

교과목 2 : 데이터 분석과 머신러닝 / 딥러닝

단원 1 : 데이터 분석

DAY3. 데이터 분석 통계

목 차

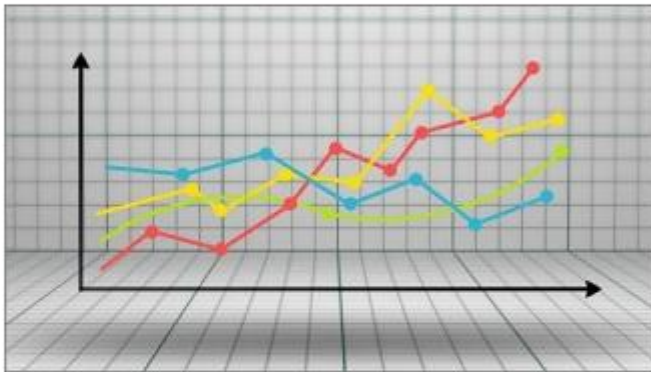
1. <u>Pandas의 자료구조</u>		3
2. <u>Pandas DataFrame</u>		16
3. <u>Pandas 통계함수</u>		31
4. <u>Pandas의 인덱스</u>		50
5. <u>Pandas의 산술 및 그룹 연산</u>		77
6. <u>Pandas 함수 정의 및 매핑</u>		94
7. <u>Pandas 데이터 전처리</u>		119
8. <u>Pandas 데이터 입출력 및 분석</u>		150

1. Pandas의 자료구조

1. Series 데이터란?

1) Series 데이터의 개요

- (1) 순차적으로 나열된 1차원 형태의 데이터
- (2) Pandas의 Series()를 이용하여 생성함
- (3) Series()의 인자로 리스트, 딕셔너리, NumPy 배열이 들어갈 수 있음
- (4) Series 객체의 인덱스는 0부터 시작함
- (5) 인덱싱 값을 지정할 수 있음(index 속성 이용)



2) Series 데이터의 특징

- (1) 레이블이 반드시 필요한 것은 아님
- (2) 정수 혹은 레이블을 기반으로 하는 인덱싱을 지원함
- (3) 통계 메서드 사용 시 자동으로 누락된 데이터(missing data, nan)를 제외시킴
- (4) 축 레이블은 인덱스와 함께 묶어서 참조할 수 있음
- (5) 서로 다른 길이의 Series 객체와의 연산이 가능함

3) Series 데이터의 활용

(1) Series 데이터 생성 방법

[형식] `pandas.Series(data, index=인덱스)`

- data
 - 여러 가지 자료형을 포함할 수 있음 (딕셔너리, 리스트, 스칼라 값, NumPy 배열)
- index
 - 축 레이블의 리스트를 할당함

3) Series 데이터의 활용

(1) Series 데이터 생성 방법

▪ 딕셔너리를 이용한 생성 예제

- import pandas as pd

```
dic={'name':['hong', 'kim', 'heo'], 'kor' : [80, 90, 75], 'eng' :  
[80, 95, 100]}
```

```
ser_data=pd.Series(dic)
```

딕셔너리 생성 후 해당 딕셔너리를
매개변수로 하여 Series 객체를 생성

```
import pandas as pd  
dic={'name':['hong', 'kim', 'heo'], 'kor' : [80, 90, 75], 'eng'  
ser_data=pd.Series(dic)  
ser_data
```

name [hong, kim, heo]
kor [80, 90, 75]
eng [80, 95, 100]
dtype: object

생성된 Series 객체의 값을 가져옴

3) Series 데이터의 활용

(1) Series 데이터 생성 방법

- NumPy 배열을 이용한 생성 예제

- `import pandas as pd`
`import NumPy as np`
`arr= np.array([80, 90, 75])`
`ser_data=pd.Series(arr)`

NumPy 배열을 생성함

```
import pandas as pd
import numpy as np
arr= np.array([80, 90, 75])
ser_data=pd.Series(arr)
ser_data
```

0	80
1	90
2	75

dtype: int64

NumPy 배열을
매개변수로 하여
Series
객체를 생성함

생성된 Series 객체의
값을 가져옴

2. DataFrame 이란?

1) DataFrame의 개요

- (1) Series 데이터들을 하나의 열로 취급한 집합
- (2) Pandas의 DataFrame()를 이용하여 생성함
- (3) 리스트, 딕셔너리, NumPy 배열, DataFrame 객체를 이용하여 생성할 수 있음
- (4) 데이터를 표 형태로 처리하는 자료구조
- (5) 인덱싱 값을 지정할 수 있음(index 속성 이용)

2) DataFrame의 특징

- (1) 서로 다른 데이터 타입의 항목을 가질 수 있음
- (2) 크기를 변경할 수 있음
- (3) 행(row)은 개별 데이터를, 열(column)은 각 데이터의 개별 속성을 의미함
- (4) 인덱스는 중복될 수 있고, 동일한 인덱스에 해당하는 행의 값이 다를 수 있음
- (5) 행과 열의 산술 연산을 수행할 수 있음

3) DataFrame의 활용

(1) DataFrame 생성 방법

[형식] `pandas.DataFrame(data, index=인덱스, columns = 레이블 목록, dtype = 데이터 타입)`

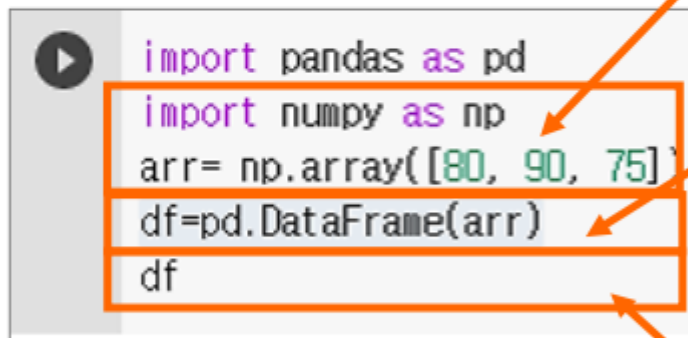
- data
 - 여러 가지 자료형을 포함할 수 있음
(딕셔너리, 리스트, 스칼라 값, NumPy 배열, DataFrame객체)
- index
 - 축(행) 레이블의 리스트를 할당함
 - 인덱스 값이 없으면 0-based index가 기본으로 사용됨
- columns
 - 열의 레이블의 리스트를 할당함
 - 인덱스 값이 없으면 0-based index가 기본으로 사용됨
- dtype
 - column마다 데이터 유형을 명기함

3) Series 데이터의 활용

(1) DataFrame 생성 방법

▪ NumPy 배열을 이용한 생성 예제

- `import pandas as pd`
`import NumPy as np`
`arr= np.array([80, 90, 75])`
`df=pd.DataFrame(arr)`



```
import pandas as pd
import numpy as np
arr= np.array([80, 90, 75])
df=pd.DataFrame(arr)
df
```

NumPy 배열을 생성함

NumPy 배열을
매개변수로 하여
DataFrame
객체를 생성함

생성된 DataFrame
객체의 값을 가져옴

»

	0
0	80
1	90
2	75

3) DataFrame의 활용

(1) DataFrame 생성 방법

- 딕셔너리를 이용한 생성 예제

- `import pandas as pd`

```
dic={'name':['hong', 'kim', 'heo'], 'kor' : [80, 90, 75], 'eng' :  
[80, 95, 100]}
```

```
df=pd.DataFrame(dic)
```

딕셔너리 생성 후 해당 딕셔너리를
매개변수로 하여 DataFrame 객체를 생성

```
import pandas as pd  
dic={'name':['hong', 'kim', 'heo'],  
      'kor' : [80, 90, 75],  
      'eng' : [80, 95, 100]}  
df=pd.DataFrame(dic)  
df
```

생성된 DataFrame
객체의 값을 가져옴



	name	kor	eng
0	hong	80	80
1	kim	90	95
2	heo	75	100

3. 색인 객체란?

1) 색인 객체의 개요

- (1) RDBMS의 기본키와 유사하게 Series 혹은 DataFrame의 데이터를 식별하는 객체
- (2) 표 형식의 데이터에서 각 행과 열의 헤더(이름)와 기타 메타데이터(축의 이름 등)를 저장하는 객체
- (3) Series와 DataFrame 생성 시, index 및 columns 속성에 할당되는 값이 색인으로 변환됨
- (4) Series와 DataFrame의 index 속성을 이용하여 추출

2) 색인 객체의 특징

- (1) 변경이 불가능한 ndarray임
- (2) 순서가 있고 슬라이스가 가능함
- (3) 미리 만들어서 안전하게 공유할 수 있음
- (4) 색인 객체의 크기 변경이 불가능함
- (5) reindex 명령을 이용한 변경은 가능
- (6) 값의 중복을 허용하기 때문에 중복된 값을 선택하면 해당 값을 가진 모든 항목이 선택됨

3) 색인 객체의 활용

(1) 색인 객체의 생성 방법

```
[형식] index객체명  
= pandas.Index(index list, dtype=None, name=None)
```

- index list
 - 인덱스 이름 목록
- dtype
 - 데이터의 유형
 - None이면 가장 적합한 데이터 유형을 찾음
- name
 - 인덱스 객체의 이름

3) 색인 객체의 활용

(2) 색인 객체 생성 후 Series 객체에 할당하기

▪ 색인 객체 만들기

- `stud_id=pd.Index(['name','kor','eng'])`

```
import pandas as pd
stud_id=pd.Index(['name','kor','eng'])
stud_id
```

Index(['name', 'kor', 'eng'], dtype='object')

▪ Series 객체 생성하기

- `stud_data=pd.Series(['hong','kim','heo'], [80,75,90], [80,90,100])`

```
stud_data=pd.Series(['hong','kim','heo'], [80,75,90], [80,90,100])
stud_data
```

0 [hong, kim, heo]
1 [80, 75, 90]
2 [80, 90, 100]
dtype: object

현재 인덱스 레이블이 없는 상태임.
시스템에서 자동으로 할당하는
인덱스 번호가 부여된 상태임

3) 색인 객체의 활용

(2) 색인 객체 생성 후 Series 객체에 할당하기

- 생성된 Series 객체에 색인 객체 할당하기
 - Series객체명.index=Index객체
- stud_data.index=stud_id

```
stud_data.index=stud_id
stud_data
```

name	[hong, kim, heo]
kor	[80, 75, 90]
eng	[80, 90, 100]
dtype: object	

인덱스 레이블이 stud_id 값으로 할당되었음

3) 색인 객체의 활용

(3) 색인 객체 관련 함수

함수	설명
append	색인 개체를 추가한 새로운 색인을 구함
insert	지정된 위치에 색인이 추가된 새로운 색인을 구함
difference	색인의 차집합을 구함
intersection	색인의 교집합을 구함
union	색인의 합집합을 구함
isin	색인이 존재하는지를 구함(불리언 배열)
drop	지정된 값이 삭제된 새로운 색인을 구함
delete	지정된 위치의 색인이 삭제된 새로운 색인을 구함
is_monotonic	색인이 단조성을 가지는지를 구함 (가지면 True)
is_unique	중복되는 색인이 있는지를 구함 (없으면 True)
unique	색인에서 중복 요소를 제거하고 유일한 값만 구함

2. Pandas DataFrame

1. DataFrame 생성하기

1) DataFrame 생성 방법

(1) Pandas를 import함

- `import pandas as pd`

(2) DataFrame 형식

- `pandas.DataFrame(data=None, index=None, columns=None, dtype=None)`

(3) Pandas의 타입

dtype	type
int64	integer
float64	decimal
object	string
bool	boolean
datetime64	date & time
category	category

1) DataFrame 생성 방법

(4) 칼럼명만 지정하여 DataFrame 생성

- `DataFrame객체명=pd.DataFrame`
`(data={'c1':[10,20,30], 'c2':[40,50,60]})`
- `DataFrame객체명=pd.DataFrame`
`({'c1':[10,20,30], 'c2':[40,50,60]})`
- `DataFrame객체명=pd.DataFrame`
`(data=[[10,40],[20,50],[30,60]], columns=['c1','c2'])`

(5) 인덱스명만 지정하여 DataFrame 생성

- `DataFrame객체명=pd.DataFrame`
`(data=[[10,40],[20,50],[30,60]], index=[1,2,3])`

(6) 칼럼명, 인덱스명을 모두 지정하여 DataFrame 생성

- `DataFrame객체명=pd.DataFrame`
`(data={'c1':[10,20,30], 'c2':[40,50,60]}, index=[1,2,3])`

1) DataFrame 생성 방법

▶ 여기서 잠깐

(1) 칼럼명, 인덱스명 없이 DataFrame을 생성할 방법은?

- Pandas와 NumPy를 import함
 - import pandas as pd
 - import numpy as np
- NumPy를 이용하여 배열을 생성함
 - arr=np.array([[10,20,30],[40,50,60]])
- 생성된 배열을 이용하여 DataFrame을 생성함
 - df=pd.DataFrame(arr)

2) NumPy를 이용한 DataFrame 생성 방법

(1) Pandas와 NumPy를 import함

- `import pandas as pd`
- `import numpy as np`

(2) NumPy를 이용하여 배열을 생성함

- `arr=np.array([[10,40],[20,50],[30,60]])`

(3) DataFrame 생성하기

- `df=pd.DataFrame`
`(data=arr, index=[1,2,3], columns=['c1', 'c2'])`

2) NumPy를 이용한 DataFrame 생성 방법

▶ 여기서 잠깐

(1) DataFrame을 df로 정의하여 사용하자.

- Pandas와 NumPy를 import함
 - `import pandas as pd`
 - `from pandas import DataFrame as df`
- 생성된 배열을 이용하여 DataFrame을 생성함
 - `a=df(data=[[10,40],[10,50],[30,60]])`

3) 딕셔너리를 이용한 DataFrame 생성 방법

(1) Pandas를 import함

- `import pandas as pd`

(2) 딕셔너리를 정의함

- 딕셔너리 정의 형식: `{key1:value1, key2:value2, ...}`
- 딕셔너리의 value에는 단일 값 및 리스트도 올 수 있음
 - `dic={'c1':[10,20,30],'c2':[40,50,60]}`

(3) DataFrame 생성함

- `df=pd.DataFrame(dic)`

2. 행과 열 추가 / 삭제하기

1) 행과 열 추가하기

(1) 행 추가하기

- DataFrame 객체명.loc[새로운 행이름]=값 혹은 배열

- 예: import pandas as pd

```
df=pd.DataFrame({'name':['hong', 'kim', 'heo'],  
                'kor' : [80, 90, 75], 'eng' : [80, 95, 100]})
```

	name	kor	eng
0	'hong'	80	80
1	'kim'	90	95
2	'heo'	75	100

- 'name' : 'song', 'kor' :95, 'eng':80인 행을 추가해 보자

- df.loc[3] = ['song', 95, 80]

	name	kor	eng
0	'hong'	80	80
...	...	...	...
3	'song'	95	80

1) 행과 열 추가하기

(2) 열 추가하기

- DataFrame 객체명[새로운 열이름]=값 혹은 배열
 - 예: `import pandas as pd`
`df=pd.DataFrame({'name':['hong', 'kim', 'heo'],
'kor' : [80, 90, 75], 'eng' : [80, 95, 100]})`
- 수학 점수 70, 85, 90을 추가해 보기
 - `df['math'] = [70, 85, 90]`

	name	kor	eng	Math
0	'hong'	80	80	70
1	'kim'	90	95	85
2	'song'	75	100	90

2) 행과 열 삭제하기

(1) 행 삭제하기

- DataFrame 객체명.drop(행 인덱스 또는 배열, axis=0, inplace=True 또는 False)
 - inplace의 값이 False로 설정되면, 원 객체를 변경하지 않고 변경된 새로운 DataFrame 객체를 생성함

(2) 열 삭제하기

- DataFrame 객체명.drop(열명 또는 배열, axis=1, inplace=True 또는 False)

3. 행과 열, 원소 선택하기

1) 행 선택하기

(1) loc를 이용한 행 선택

- 1개의 행 선택
 - DataFrame 객체명.loc[선택할 행이름]
- 범위지정 선택
 - DataFrame 객체명.loc[시작행이름:끝행이름]
 - 범위의 끝을 포함함

(2) iloc를 이용한 행 선택

- 1개의 행 선택
 - DataFrame 객체명.iloc[선택할 인덱스]
- 범위지정 선택
 - DataFrame 객체명.iloc[시작인덱스:끝인덱스:슬라이싱간격]
 - 범위의 끝을 제외함
 - 슬라이싱 간격은 생략할 수 있으며, 생략할 경우 1로 간주함
 - 슬라이싱 간격을 -1로 지정하면 역순으로 제시됨
 - 모든 행을 선택할 경우에는 시작인덱스와 끝인덱스를 지정하지 않음

2) 열 선택하기

(1) 1개의 열 선택하기

- DataFrame 객체명[선택할 열이름]

(2) 여러 개의 열 선택하기

- DataFrame 객체명[[선택할 열이름1, 선택할 열이름2,...]]

- 예: import pandas as pd

```
df=pd.DataFrame({'name':['hong', 'kim', 'heo'],
```

```
'kor' : [80, 90, 75], 'eng' : [80, 95, 100]})
```

```
eng_data=df['eng']
```

```
name_eng_data=df[['name', 'eng']]
```

3) 원소 선택하기

(1) loc를 이용한 원소 선택

- DataFrame 객체명.loc[행 인덱스 혹은 행명, 열 인덱스 혹은 열명]

(2) iloc를 이용한 원소 선택

- DataFrame 객체명.iloc[행 번호, 열 번호]

3) 원소 선택하기

(3) iloc를 이용한 원소 선택

- DataFrame 객체명.iloc[행 번호, 열 번호]

- 예: import pandas as pd

```
df=pd.DataFrame({'name':['hong', 'kim', 'heo'], 'kor' :  
[80, 90, 75], 'eng' : [80, 95, 100]})
```

```
hong_eng_data1=df.loc[0, 'eng']
```

```
hong_eng_data1=df.iloc[0, 2]
```

```
name_eng_data=df.loc[1, ['eng', 'kor']]
```

```
name_eng_data=df.iloc[1, [2, 1]]
```

hong의 영어 성적을 선택함(80)

kim의 영어와 수학 성적을 선택함 (95, 90)

	name	kor	eng
0	'hong'	80	80
1	'kim'	90	95
2	'heo'	75	100

4. 위치 변경하기

1) 행 위치 변경하기

(1) reindex를 이용하여 변경함

[형식] DataFrame 객체명.reindex(index=[인덱스목록])

▪ 예: import pandas as pd

```
df=pd.DataFrame({'name':['hong', 'kim', 'heo'],
```

```
'kor' : [80, 90, 75], 'eng' : [80, 95, 100]})
```

```
df=df.reindex(index=[2,0,1])
```



	name	kor	eng
2	'heo'	75	100
0	'hong'	80	80
1	'kim'	90	95

	name	kor	eng
0	'hong'	80	80
1	'kim'	90	95
2	'heo'	75	100

2) 열 위치 변경하기

(1) 대괄호 안에 원하는 순서대로 칼럼명을 나열함

▪ 예: import pandas as pd

```
df=pd.DataFrame({'name':['hong', 'kim', 'heo'], 'kor' :  
[80, 90, 75], 'eng' : [80, 95, 100]})
```

```
df=df[['name', 'eng', 'kor']]
```



	name	eng	kor
0	'hong'	80	80
1	'kim'	95	90
2	'heo'	100	75

	name	kor	eng
0	'hong'	80	80
1	'kim'	90	95
2	'heo'	75	100

3) 행과 열 위치 변경하기

(1) DataFrame 객체명.transpose() 혹은 DataFrame 객체명.T

▪ 예: import pandas as pd

```
df=pd.DataFrame({'name':['hong', 'kim', 'heo'],
```

```
'kor' : [80, 90, 75], 'eng' : [80, 95, 100]})
```

```
df=df.transpose( ) # df=df.T
```



	0	1	2
Name	'hong'	'kim'	'heo'
Eng	80	90	75
kor	80	95	100

	name	kor	eng
0	'hong'	80	80
1	'kim'	90	95
2	'heo'	75	100

3. Pandas 통계함수

1. 평균값, 중간값, 최솟값, 최댓값 익히기

1) 평균값, 중간값, 최솟값, 최댓값 소개

(1) 평균값이란?

- 중심경향치 중의 하나(자료 전체를 대표 : 평균값, 중앙값, 최빈값 등)
- 데이터의 총합을 총 개수로 나눈 값
- 특정 열을 선택하여 계산할 수도 있음
- 극단적인 값의 영향을 받음
- 비대칭 분포 데이터의 경우, 전체 자료를 대표하기가 어려움
- 데이터 중 문자열 또는 문자는 무시하고 계산함
- mean

(2) 중간값(중앙값 또는 중위수)이란?

- 어떤 주어진 값들을 크기 순으로 정렬했을 때 가장 중앙에 위치하는 값
- 자료의 상위 50%와 하위 50%를 가르는 지점
- 특정 열을 선택하여 계산할 수도 있음
- 데이터 중 문자열 또는 문자는 무시하고 계산함
- 총 데이터 수가 짝수일 경우에는 2개의 중앙값을 가짐
- median

1) 평균값, 중간값, 최솟값, 최댓값 소개

(3) 최빈값이란?

- 최빈값: mode
 - 가장 빈번하게 관찰/측정되는 값
 - 주로 범주변수(명명척도, 서열척도)에 사용됨

(4) 최솟값이란?

- 전체 데이터 값 중 최솟값을 구함
- 특정 열을 선택하여 계산할 수도 있음
- 데이터 중 문자열 또는 문자는 무시하고 계산함
- min
- ▶ 특정 열이 최솟값을 가지는 행 추출: idmin

[형식] DataFrame 객체

.idmin(axis = 적용할 축, skipna=True(기본값)/false)

- 최솟값을 가지는 인덱스 레이블을 추출
- skipna이 True이면, Na/null값들을 제외함

1) 평균값, 중간값, 최솟값, 최댓값 소개

(5) 최댓값이란?

- 전체 데이터 값 중 최댓값을 구함
- 특정 열을 선택하여 계산할 수도 있음
- 데이터 중 문자열 또는 문자는 무시하고 계산함
- max
- ▶ 특정 열이 최댓값을 가지는 행 추출: idmax

[형식] DataFrame 객체.idmax(axis = 적용할 축,
skipna=True(기본값)/false)

- 최댓값을 가지는 인덱스 레이블을 추출
- skipna이 True이면, Na/null값들을 제외함

2) 평균값, 중간값 구하기

(1) 평균값 구하는 함수

- [형식] DataFrame 객체.mean(axis=None, skipna=None)
 - axis에 설정된 행(0) 또는 열(1)을 따라 평균값을 찾음(기본값 0)
 - skipna에는 Na/null값을 제외할 것인지를 설정(기본값 True)
- 주어진 DataFrame 객체의 열을 따라 평균값을 구하기(None 제외)

```
import pandas as pd

df = pd.DataFrame({'X': [1, 2, None, 3],
                  'Y': [4, 3, 8, 4]})

df.mean()
```

X	2.00
Y	4.75
dtype: float64	

2) 평균값, 중간값 구하기

(1) 평균값 구하는 함수

- 주어진 DataFrame 객체의 열을 따라 평균값을 구하기(None 포함)

```
import pandas as pd

df = pd.DataFrame({'X': [1, 2, None, 3],
                   'Y': [4, 3, 8, 4]})
df.mean(skipna=False)
```

X	NaN
Y	4.75
dtype: float64	

- 주어진 DataFrame 객체의 행을 따라 평균값을 구하기(None 제외)

```
import pandas as pd

df = pd.DataFrame({'X': [1, 2, None, 3],
                   'Y': [4, 3, 8, 4]})
df.mean(axis=1)
```

0	2.5
1	2.5
2	8.0
3	3.5
dtype: float64	

2) 평균값, 중간값 구하기

(2) 중간값 구하는 함수

- [형식] DataFrame 객체.median(axis=None, skipna=None)
 - axis에 설정된 행(0) 또는 열(1)을 따라 중간값을 찾음(기본값 0)
 - skipna에는 Na/null값을 제외할 것인지를 설정(기본값 True)
- 주어진 DataFrame 객체의 열을 따라 중간값을 구하기(None 제외)

```
import pandas as pd

df = pd.DataFrame({'x': [1, 2, None, 3],
                   'y': [4, 3, 8, 4]})

df.median()
```

```
x    2.0
y    4.0
dtype: float64
```

3) 최솟값, 최댓값 구하기

(1) 최솟값 구하는 함수

- [형식] DataFrame 객체.min(axis=None, skipna=None)
 - axis에 설정된 행(0) 또는 열(1)을 따라 최솟값을 찾음(기본값 0)
 - skipna에는 Na/null값을 제외할 것인지를 설정(기본값 True)
- 주어진 DataFrame 객체의 열을 따라 최솟값을 구해보자.

(None 제외)

```
import pandas as pd

df = pd.DataFrame({'X': [1, 2, None, 3],
                   'Y': [4, 3, 8, 4]})

df.min()
```

X	1.0
Y	3.0
dtype: float64	

3) 최솟값, 최댓값 구하기

(2) 최댓값 구하는 함수

- [형식] DataFrame 객체.max(axis=None, skipna=None)
 - axis에 설정된 행(0) 또는 열(1)을 따라 최댓값을 찾음(기본값 0)
 - skipna에는 Na/null값을 제외할 것인지를 설정(기본값 True)
- 주어진 DataFrame 객체의 열을 따라 최댓값을 구해보자.

(None 제외)

```
import pandas as pd

df = pd.DataFrame({'X': [1, 2, None, 3],
                   'Y': [4, 3, 8, 4]})

df.max()
```

```
X    3.0
Y    8.0
dtype: float64
```

2. 표준편차, 분산, 공분산 익히기

1) 표준편차, 분산, 공분산 소개

(1) 표준편차란?

- 자료가 평균을 중심으로 얼마나 퍼져 있는지를 나타내는 대표적인 수치
- 각각의 데이터가 평균에서 얼마나 벗어나는지를 나타냄
- 표준편차가 0에 가까우면 데이터가 평균 근처에 집중되어 있음을 의미
- 표준편차가 클수록 데이터들은 널리 퍼져 있음
- std

(2) 분산과 공분산이란?

- 분산이란?
 - 값들이 흩어진 정도를 계산하는 지표
 - 평균과의 차이를 제곱한 것
 - var
- 공분산이란?
 - 두 변수의 상관 정도를 나타내는 값
 - 공분산이 양수이면, 두 변수가 같이 상승하는 경향을 보임
 - cov

2) 표준편차, 분산, 공분산 구하기

(1) 표준편차 관련 함수

- [형식] DataFrame 객체.std(axis=None, skipna=None)
 - axis에 설정된 행(0) 또는 열(1)을 따라 표준편차를 구함
 - skipna에는 Na/null값을 제외할 것인지를 설정(기본값 True)

```
import pandas as pd
df=pd.DataFrame({'X': [1,2,None,3],
                 'Y': [4,3,8,4]})
df.std()
```

X	1.000000
Y	2.217356
dtype: float64	

2) 표준편차, 분산, 공분산 구하기

(2) 분산 관련 함수

- [형식] DataFrame 객체.var(axis=None, skipna=None)
 - axis에 설정된 행(0) 또는 열(1)을 따라 분산을 구함(기본값 0)
 - skipna에는 Na/null값을 제외할 것인지를 설정(기본값 True)



```
import pandas as pd
df=pd.DataFrame({'X': [1, 2, None, 3],
                  'Y': [4, 3, 8, 4]})
df.var()
```



```
X    1.000000
Y    4.916667
dtype: float64
```

2) 표준편차, 분산, 공분산 구하기

(3) 공분산 관련 함수

- [형식] DataFrame 객체.cov(axis=None, skipna=None)
 - axis에 설정된 행(0) 또는 열(1)을 따라 공분산을 구함(기본값 0)
 - skipna에는 Na/null값을 제외할 것인지를 설정(기본값 True)



```
import pandas as pd
df=pd.DataFrame({'X': [1,2,None,3],
                 'Y': [4,3,8,4]})
df.cov()
```



	X	Y
X	1.0	0.000000
Y	0.0	4.916667

3. 상관계수 익히기

1) 상관계수 소개

(1) 상관계수란?

- 두 개의 연속형 변수(열) 간의 연관성에 대한 측도
- 상관계수는 두 변수 간의 연관된 정도를 나타낼 뿐 인과관계를 나타내는 것은 아님
- 상관계수의 종류는 피어슨(Pearson), 켄달(Kendall), 스피어만(Spearman)가 있음
- 가장 대표적으로 사용되는 상관계수는 피어슨임
- -1~1 사이의 값을 가지며, 양수일 경우 양의 상관관계, 음수일 경우 음의 상관관계(부적상관)이라 함
- corr

(2) 상관계수 종류

- 피어슨(Pearson) 상관계수
 - 가장 많이 활용됨
 - 두 변수(열) 간 선형적인 관계(선형성 : linearity)가 있는지 판단하는 데 사용
 - 즉, 0에 가깝더라도 선형적인 관계가 낮을 뿐이지, 서로 유의미한 관계를 갖지 않는다는 것은 아님
 - -1에 가까울수록 음의 선형적인 상관관계를 가지고, 1에 가까울수록 양의 선형적인 상관관계를 가짐

1) 상관계수 소개

(2) 상관계수 종류

- 스피어만(Spearman) 상관계수: 순위상관계수
 - 피어슨 상관계수와 같은 원리로 산출
 - 단조성(monotonicity)을 평가하기 위해 사용
 - 두 연속 변수가 정규분포를 심각하게 벗어나거나 순위척도 자료일 때 사용
 - 데이터의 순서통계량 값을 활용
 - 즉, 두 변수의 값을 각각 제일 큰 값을 1, 두 번째 큰 값을 2, ...로 변환한 후 변환된 값들 간의 피어슨 상관계수를 구함
 - ▶ 데이터의 순서통계량 값 + 피어슨 상관계수 원리
→ 데이터의 순서통계량 값
- 켄달(Kendall) 상관계수: 순위상관계수
 - 두 변수 간의 순위를 비교하여 연관성을 계산
 - 두 변수(열) 간의 방향을 분석하여 산출
 - 단조성을 판단하기 위해 사용
 - 두 연속 변수가 정규분포를 심각하게 벗어나거나 순위척도 자료일 때 사용
 - 스피어만의 상관계수와 비슷한 방식으로 순위척도 자료 또는 순위척도로 변환한 자료를 이용

1) 상관계수 소개

(2) 상관계수 종류

- 켄달(Kendall) 상관계수: 순위상관계수
 - 한 변수가 증가할 때, 다른 변수의 증가하는 횟수와 감소하는 횟수를 분석하여 상관계수로 표현

2) 상관계수 구하기

- (1) [형식] `DataFrame 객체.corr(method='pearson', min_periods=1)`
- `method`에는 상관의 방법을 명기함
 - 피어슨, 켄달 및 스피어만을 설정할 수 있음(기본값은 피어슨)
 - `min_periods`는 유효한 결과를 얻기 위해 열 쌍당 필요한 최소 관측치 수를 지정(현재, 피어슨과 스피어만 상관계수에서만 사용)

2) 상관계수 구하기

(2) 피어슨(Pearson) 상관계수 구하기

- 국어와 영어 성적 간의 피어슨 상관계수를 구하자.

```
import pandas as pd
df=pd.DataFrame({'name':['hong','kim','heo'],
                 'kor':[80, 90, 75],
                 'eng':[80, 95, 100]})

df
```

	name	kor	eng
0	hong	80	80
1	kim	90	95
2	heo	75	100

```
df.corr()
```

	kor	eng
kor	1.000000	-0.052414
eng	-0.052414	1.000000

```
df.corr(method='pearson')
```

	kor	eng
kor	1.000000	-0.052414
eng	-0.052414	1.000000

2) 상관계수 구하기

(3) 스피어만(Spearman) 상관계수 구하기

- 국어와 영어 성적 간의 스피어만 상관계수를 구하자.

```
import pandas as pd
df=pd.DataFrame({'name':['hong', 'kim', 'heo'],
                 'kor':[80, 90, 75],
                 'eng':[80, 95, 100]})

df
```

	name	kor	eng
0	hong	80	80
1	kim	90	95
2	heo	75	100

```
df.corr(method='spearman')
```

	kor	eng
kor	1.0	-0.5
eng	-0.5	1.0

```
df.corr(method='spearman', min_periods=2)
```

	kor	eng
kor	1.0	-0.5
eng	-0.5	1.0

2) 상관계수 구하기

(4) 켄달(Kendall) 상관계수 구하기

- 국어와 영어 성적 간의 켄달 상관계수를 구하자.

```
import pandas as pd
df=pd.DataFrame({'name':['hong','kim','heo'],
                 'kor':[80, 90, 75],
                 'eng':[80, 95, 100]})
df
```

	name	kor	eng
0	hong	80	80
1	kim	90	95
2	heo	75	100



```
df.corr(method='kendall')
```

	kor	eng
kor	1.000000	-0.333333
eng	-0.333333	1.000000

2) 상관계수 구하기

(4) 켄달(Kendall) 상관계수 구하기

- 국어와 영어 성적 간의 켄달 상관계수를 구하자.

```
import pandas as pd
df=pd.DataFrame({'name':['hong','kim', 'heo'],
                 'kor' : [60, 80, 95],
                 'eng' : [80, 95, 100]})
df
```

	name	kor	eng
0	hong	60	80
1	kim	80	95
2	heo	95	100



```
df.corr(method='kendall')
```

	kor	eng
kor	1.0	1.0
eng	1.0	1.0

4. Pandas의 인덱스

1. 인덱스란?

1) 인덱스의 특징

- (1) DataFrame, Series의 레코드를 고유하게 식별하는 객체
- (2) 식별성 데이터를 1차원 배열로 가지고 있음
- (3) 크기 변경이 불가능하고 중복된 값을 가질 수 있음
- (4) 단일값 반환 및 슬라이싱 가능
- (5) 인덱스 객체의 값은 "인덱스객체[인덱스번호]=값"처럼 수정할 수 없음

2) 인덱스의 종류

(1) 명칭 기반 인덱싱

- 칼럼의 명칭을 기반으로 위치를 지정하는 방식
- "칼럼명"과 같이 명칭으로 열 위치를 지정
- .loc를 이용하면 명칭 기반 인덱싱으로 인덱스의 이름과 칼럼의 이름으로 접근 가능
- 명칭 기반으로 하나의 스칼라 값을 뽑을 때는 at을 이용
 - [형식] DataFrame객체.loc[행 인덱싱 값] 또는 DataFrame객체.loc[행 인덱싱 값, 열 인덱싱 값]

2) 인덱스의 종류

(2) 위치 기반 인덱싱

- 0을 시작 값으로 행과 열의 위치를 지정
- 행과 열 인덱싱 값으로는 정수가 옴
- .iloc를 이용하면 위치 기반 인덱싱으로 인덱스(숫자)로만 접근 가능
- 위치 기반으로 하나의 스칼라 값을 뽑을 때는 iat을 이용
 - [형식] DataFrame객체.iloc[행 인덱싱 값] 또는 DataFrame객체.iloc[행 인덱싱 값, 열 인덱싱 값]

(3) 불리언 인덱싱

- [] 안에 조건식을 입력하여 해당하는 데이터만 추출할 때 사용
- loc[], iloc[]에서도 불리언 인덱싱 적용 가능
 - [형식] DataFrame객체[조건식] 또는 DataFrame객체.loc[조건식] 또는 DataFrame객체.iloc[조건식]

3) 인덱스 사용하기

(1) 명칭 기반 인덱싱 사용하기

- 아래와 같은 데이터를 저장하기 위한 DataFrame을 생성하고 "name"을 추출

Name	kor	eng	mat
홍길동	80	90	85
이기자	70	75	85
최신	100	90	95

```
▶ df["name"]  
  
1    홍길동  
2    이기자  
3    최신  
Name: name, dtype: object
```

```
▶ df.loc[[1,2,3], "name"]  
  
1    홍길동  
2    이기자  
3    최신  
Name: name, dtype: object
```

```
▶ import pandas as pd  
df = pd.DataFrame(  
    data=[  
        ["홍길동", 80, 90, 85],  
        ["이기자", 70, 75, 85],  
        ["최신", 100, 90, 95]  
    ],  
    index=[1, 2, 3],  
    columns=["name", "kor", "eng", "mat"]  
)
```

3) 인덱스 사용하기

(1) 명칭 기반 인덱싱 사용하기

- 이름과 영어 성적만 추출
 - `DataFrame명[["name", "eng"]]`



```
df[["name", "eng"]]
```

	name	eng
1	홍길동	90
2	이기자	75
3	최신	90



```
df.loc[[1,2,3],["name", "eng"]]
```

	name	eng
1	홍길동	90
2	이기자	75
3	최신	90

3) 인덱스 사용하기

(2) 위치 기반 인덱싱 사용하기

- `iloc` 이용
 - 1개의 행 지정: `DataFrame명.iloc[행인덱스]`
 - 특정 행과 열 위치 지정: `DataFrame명.iloc [행인덱스, 열인덱스]`
- ▶ 지정할 행과 열이 하나 또는 연속 인덱스가 아닌 경우, `[]`로 묶어 줌
 - 예: 1~3의 경우는 `1:3`으로 표현하나, 1 또는 3의 경우는 `[1,3]`으로 표현
 - 특정 행 지정: `DataFrame명.iloc [시작행인덱스:끝행인덱스+1]`

(2) 위치 기반 인덱싱하기

- 아래와 같은 데이터를 저장하기 위한 DataFrame을 생성하고 첫 번째 데이터를 추출

Name	kor	eng	mat
홍길동	80	90	85
이기자	70	75	85
최신	100	90	95

▶ df[0:1]

	name	kor	eng	mat
1	홍길동	80	90	85

▶ df.iloc[0]

name	홍길동
kor	80
eng	90
mat	85

Name: 1, dtype: object

▶

```
import pandas as pd
df = pd.DataFrame(
    data=[
        ["홍길동", 80, 90, 85],
        ["이기자", 70, 75, 85],
        ["최신", 100, 90, 95]
    ],
    index=[1, 2, 3],
    columns=["name", "kor", "eng", "mat"]
)
```

3) 인덱스 사용하기

(2) 위치 기반 인덱싱하기

- 첫 번째 ~ 두 번째 데이터의 성명과 영어만 추출

- DataFrame명

.iloc[시작index:끝index+1, [성명의 index, 영어의 index]]



The image shows a Jupyter Notebook cell with a code input and a resulting DataFrame output. The code input is `df.iloc[0:2, [0,2]]`. The output is a DataFrame with two columns, 'name' and 'eng', and two rows of data.

	name	eng
1	홍길동	90
2	이기자	75

(3) 불리언 인덱싱하기

- [] 안에 조건식을 입력하여 해당하는 데이터만 추출할 때 사용
- loc[], iloc[]에서도 불리언 인덱싱 적용 가능
 - [형식] DataFrame객체[조건식] 또는
DataFrame객체.loc[조건식] 또는
DataFrame객체.iloc[조건식]

3) 인덱스 사용하기

(3) 불리언 인덱싱하기

- 아래와 같은 데이터를 저장하기 위한 DataFrame을 생성하고
영어 성적이 90 이상인 데이터만 추출

Name	kor	eng		mat
홍길동	80	90	True	85
이기자	70	75	False	85
최신	100	90	True	95

```
import pandas as pd
df = pd.DataFrame(
    data=[
        ["홍길동", 80, 90, 85],
        ["이기자", 70, 75, 85],
        ["최신", 100, 90, 95]
    ],
    index=[1, 2, 3],
    columns=["name", "kor", "eng", "mat"]
)
```

3) 인덱스 사용하기

(3) 불리언 인덱싱하기

- 아래와 같은 데이터를 저장하기 위한 DataFrame을 생성하고
영어 성적이 90 이상인 데이터만 추출

① eng가 90 이상인 조건을 만족하는 행을 구하기

- `df.eng>=90`

```
df.eng>=90
```

1	True
2	False
3	True

Name: eng, dtype: bool

3) 인덱스 사용하기

(3) 불리언 인덱싱하기

- 아래와 같은 데이터를 저장하기 위한 DataFrame을 생성하고
영어 성적이 90 이상인 데이터만 추출

② eng가 90 이상인 데이터의 행의 논리값을 이용하여 데이터를 추출

- `df.eng>=90`

Name	kor	eng	mat	
홍길동	80	90	85	True
이기자	70	75	85	False
최신	100	90	95	True

```

▶ df[df.eng>=90]

```

	name	kor	eng	mat
1	홍길동	80	90	85
3	최신	100	90	95

3) 인덱스 사용하기

(3) 불리언 인덱싱하기

- 영어 성적이 90 이상인 데이터의 이름과 영어만 추출

- loc 함수 이용

① eng가 90 이상인 데이터의 행을 추출

- `df.loc[df.eng>=90]`

② ①에서 추출한 결과에서 이름과 영어만 추출

- `df.loc[df.eng>=90, ["name","eng"]]`



	name	kor	eng	mat
1	홍길동	80	90	85
3	최신	100	90	95

2. 인덱스 설정하기

1) DataFrame의 인덱스 배정

(1) index 속성

- DataFrame에 인덱스를 배정할 수 있음
- DataFrame에 배정된 인덱스를 반환하는 역할도 수행함
 - [형식] DataFrame객체.index = 배열

(2) 인덱스 배정하기

- DataFrame 시 인덱스를 설정하지 않으면 자동으로 배정됨

① 인덱스를 설정하지 않고 DataFrame을 생성함

```
import pandas as pd
df=pd.DataFrame(
data=[
["홍길동",80,90,85],
["이기자",70,75,85],
["최신",100,90,95]
],
columns=["name","kor","eng","mat"]
)
```



	name	kor	eng	mat
0	홍길동	80	90	85
1	이기자	70	75	85
2	최신	100	90	95

② index 속성을 이용하여 DataFrame에 배정된 인덱스를 확인

```
df.index
RangeIndex(start=0, stop=3, step=1)
```

1) DataFrame의 인덱스 배정

(2) 인덱스 배정하기

- 생성된 DataFrame에 1, 2, 3이라는 인덱스를 배정
 - DataFrame객체.index=[1,2,3]

df

	name	kor	eng	mat
0	홍길동	80	90	85
1	이기자	70	75	85
2	최신	100	90	95

df.index=[1,2,3]
df

	name	kor	eng	mat
1	홍길동	80	90	85
2	이기자	70	75	85
3	최신	100	90	95

2) 칼럼명 인덱스로 설정하기

(1) set_index 함수

- 특정 열을 행 인덱스로 설정해 줌
- inplace가 True이면, 인덱스 열이 수정된 DataFrame을 반환하지만, 그렇지 않으면 None을 반환
 - [형식] DataFrame객체.set_index
(keys, drop=True, append=False, inplace=False)y

매개변수	설명
keys	인덱스로 설정할 열 또는 열 목록
drop	Bool. 기본값은 True로 인덱스로 설정할 칼럼을 삭제
append	Bool. 기본값은 False이며 기존 색인에 열을 추가할를 지정
inplace	Bool. True이면 호출자 DataFrame을 제자리에서 수정

2) 칼럼명 인덱스로 설정하기

(2) name을 행 인덱스로 설정

① 다음 아래의 DataFrame을 생성한 후 출력

- 인덱스가 숫자로 설정된 것을 확인할 수 있음

```
import pandas as pd
df=pd.DataFrame(
    data=[
        ["홍길동",80,90,85],
        ["이기자",70,75,85],
        ["최신",100,90,95]
    ],
    index=[1,2,3],
    columns=["name","kor","eng","mat"]
)
```

	name	kor	eng	mat
1	홍길동	80	90	85
2	이기자	70	75	85
3	최신	100	90	95

2) 칼럼명 인덱스로 설정하기

(2) name을 행 인덱스로 설정

② 숫자로 설정된 행 인덱스를 성명으로 변경

- `df.set_index("name")`
- 숫자로 설정되었던 인덱스가 이름으로 바뀐 것을 확인할 수 있음



```
df.set_index("name")
```

	kor	eng	mat
name			
홍길동	80	90	85
이기자	70	75	85
최신	100	90	95

3) 인덱스 초기화하기

(1) reset_index 함수

- 인덱스 값들을 DataFrame의 열로 전송하며,
단순 정수 인덱스로 초기화함
 - [형식] DataFrame객체.reset_index (drop=True,
inplace=False)

매개변수	설명
drop	Bool. 기본값은 True로 인덱스로 설정된 열을 DataFrame 내에서 삭제할지를 결정
inplace	Bool. 원본 객체를 변경할지를 결정

3) 인덱스 초기화하기

(2) 인덱스를 DataFrame으로 전송하지 않고, 인덱스 초기화하기

- DataFrame객체.reset_index(drop=True)

① DataFrame을 생성

- import pandas as pd

```
df=pd.DataFrame(  
    data=[  
        ["홍길동",80,90,85],  
        ["이기자",70,75,85],  
        ["최신",100,90,95]  
    ],  
    columns=["name","kor","eng","mat"]  
)
```



	name	kor	eng	mat
0	홍길동	80	90	85
1	이기자	70	75	85
2	최신	100	90	95

3) 인덱스 초기화하기

(2) 인덱스를 DataFrame으로 전송하지 않고, 인덱스 초기화하기

- DataFrame객체.reset_index(drop=True)

② DataFrame 객체의 인덱스를 초기화

- df.reset_index(drop=True)



```
df.reset_index(drop=True)
```

	name	kor	eng	mat
0	홍길동	80	90	85
1	이기자	70	75	85
2	최신	100	90	95

3) 인덱스 초기화하기

▶ 여기서 잠깐

(1) `df.reset_index()`의 실행 결과는?

	kor	eng	mat
name			
홍길동	80	90	85
이기자	70	75	85
최신	100	90	95



▶ <code>df.reset_index()</code>					
	index	name	kor	eng	mat
0	0	홍길동	80	90	85
1	1	이기자	70	75	85
2	2	최신	100	90	95

3. 인덱스 재배열 및 정렬하기

1) 인덱스 재배열하기

(1) reindex 함수

- 기본적인 정렬 함수
- 특정 행을 따라 데이터의 순서를 조정함
- 행 인덱스를 새로운 배열로 재지정해 줌
 - [형식] DataFrame객체.reindex (새로운 인덱스 배열, fill_value=값)
- reindex 수행 순서
 - 기존 데이터 인덱스를 새로운 세트에 매치시켜 순서를 재조정
 - 인덱스는 존재하나, 데이터가 없는 인덱스에 누락 값을 채움

1) 인덱스 재배열하기

(2) 다음 아래의 DataFrame의 인덱스를 2, 1, 3순으로 재조정

- `df.reindex([2,1,3])`

```
import pandas as pd
df=pd.DataFrame(
    data=[
        ["홍길동", 80, 90, 85],
        ["이기자", 70, 75, 85],
        ["최신", 100, 90, 95]
    ],
    index=[1, 2, 3],
    columns=["name", "kor", "eng", "mat"]
)
```



```
df.reindex([2,1,3])
```

	name	kor	eng	mat
2	이기자	70	75	85
1	홍길동	80	90	85
3	최신	100	90	95

1) 인덱스 재배열하기

(3) 기존 인덱스에 4, 5를 추가하여 2, 3, 4, 5순으로 재조정

- `df.reindex([2,3,4,5])`

▶ `df.reindex([2, 3, 4, 5])`

	name	kor	eng	mat
2	이기자	70.0	75.0	85.0
3	최신	100.0	90.0	95.0
4	NaN	NaN	NaN	NaN
5	NaN	NaN	NaN	NaN

(4) 기존 인덱스에 4, 5를 추가하여 2, 3, 4, 5순으로 재조정
(단, 새로운 인덱스에 0으로 값을 할당함)

- `df.reindex([2,3,4,5], fill_value=0)`

▶ `df.reindex([2, 3, 4, 5], fill_value=0)`

↳

	name	kor	eng	mat
2	이기자	70	75	85
3	최신	100	90	95
4	0	0	0	0
5	0	0	0	0

2) 인덱스 정렬하기

(1) 행 기준 정렬

- `sort_index` 함수

- 행 인덱스를 기준으로 DataFrame 값을 정렬해 줌

[형식] `DataFrame객체.sort_index(ascending=True)`

- 오름차순으로 정렬

- `df.sort_index()` 또는 `df.sort_index(ascending=True)`

▶	<code>df.sort_index()</code>				
↳	name	kor	eng	mat	
1	홍길동	80	90	85	
2	이기자	70	75	85	
3	최신	100	90	95	

▶	<code>df.sort_index(ascending=True)</code>				
↳	name	kor	eng	mat	
1	홍길동	80	90	85	
2	이기자	70	75	85	
3	최신	100	90	95	

2) 인덱스 정렬하기

(1) 행 기준 정렬

- 내림차순으로 정렬

- `df.sort_index(ascending=False)`

	<code>df.sort_index(ascending=False)</code>			
	<code>name</code>	<code>kor</code>	<code>eng</code>	<code>mat</code>
<code>3</code>	<code>최신</code>	<code>100</code>	<code>90</code>	<code>95</code>
<code>2</code>	<code>이기자</code>	<code>70</code>	<code>75</code>	<code>85</code>
<code>1</code>	<code>홍길동</code>	<code>80</code>	<code>90</code>	<code>85</code>

2) 인덱스 정렬하기

(2) 열 기준 정렬

- `sort_values` 함수
 - 열 인덱스를 기준으로 DataFrame 값을 정렬해 줌

[형식] DataFrame객체.sort_values
(필드리스트, ascending=True)

- 영어성적을 오름차순으로 정렬
 - `df.sort_values("eng")`



The screenshot shows a Jupyter Notebook cell with the code `df.sort_values("eng")` and its output. The output is a DataFrame with columns `name`, `kor`, `eng`, and `mat`, sorted by the `eng` column in ascending order.

	name	kor	eng	mat
2	이기자	70	75	85
1	홍길동	80	90	85
3	최신	100	90	95

2) 인덱스 정렬하기

(2) 열 기준 정렬

- 영어, 수학 성적을 내림차순으로 정렬
 - df.sort_values(["eng","mat"], ascending=False)



```
df.sort_values(["eng","mat"], ascending=False)
```

	name	kor	eng	mat
3	최신	100	90	95
1	홍길동	80	90	85
2	이기자	70	75	85

5. Pandas의 산술 및 그룹 연산

1. Series 데이터 산술 연산하기

1) Series와 숫자 간 연산하기

(1) Series와 숫자 간 연산 규칙

- Series의 개별 원소의 숫자를 각각 연산
- 덧셈, 뺄셈, 곱셈, 나눗셈 모두 가능
- NaN 값을 가진 요소의 연산 결과는 NaN으로 반환

(2) Series와 숫자 간 연산 형식

- Series 객체 + 연산자(사칙연산) + 숫자

(3) 수학 점수를 20점 만점으로 환산하기

```
import pandas as pd
score=pd.Series({'kor':90, 'eng':100, 'math':85, 'sec':96})
print(score)
Conversion_score=score*0.2
print('\n')
print(Conversion_score)
```

kor	90
eng	100
math	85
sec	96
dtype: int64	



kor	18.0
eng	20.0
math	17.0
sec	19.2
dtype: float64	

2) df.sort_index(ascending=False)

(1) Series 간 연산 방법

- 두 Series 객체 간의 연산은 동일한 index끼리 연산 수행
- Series 객체의 크기 또는 index가 일치하지 않는 경우에는 연산 불가

kor	eng	mat
80	90	85

eng	kor	mat
70	75	85

(2) 크기가 다른 경우

- 존재하지 않는 index는 NaN으로 채움

kor	eng	mat
80	90	85

eng	kor
70	75

kor	eng	mat
160.0	155.0	NaN

(3) 두 Series 객체의 덧셈 연산 수행하기(동일 크기)

```
import pandas as pd
score1=pd.Series({'kor':80, 'eng':90, 'math':85})
score2=pd.Series({'eng':70, 'kor':75, 'math':85})
res=score1+score2
print(res)
```

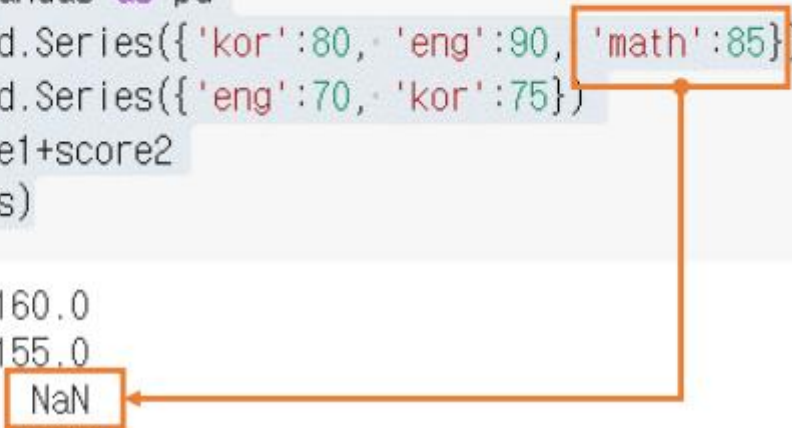
```
eng    160
kor    155
math    170
dtype: int64
```

(4) 두 Series 객체의 덧셈 연산 수행하기(다른 크기)

- 존재하지 않는 index는 NaN으로 채움

```
import pandas as pd
score1=pd.Series({'kor':80, 'eng':90, 'math':85})
score2=pd.Series({'eng':70, 'kor':75})
res=score1+score2
print(res)
```

```
eng    160.0
kor    155.0
math    NaN
dtype: float64
```



3) Series 사칙연산 함수

(1) 사칙연산 함수

덧셈	뺄셈	곱셈	나눗셈
add	sub	mul	div

(2) 두 객체 사이에 공통된 인덱스가 없거나 NaN이 포함된 경우,
NaN을 반환

(3) NaN이 반환되는 것을 방지: fill_value

(4) 서로 크기가 다른 두 Series 객체의 덧셈 연산 수행하기
(add 함수 이용)

```
import pandas as pd
score1=pd.Series({'kor':80, 'eng':90, 'math':85})
score2=pd.Series({'eng':70, 'kor':75})
res=score1.add(score2, fill_value=0)
print(res)
```

```
eng      160.0
kor      155.0
math      85.0
dtype: float64
```

2. DataFrame 산술 연산하기

1) DataFrame과 숫자 간 연산하기

(1) DataFrame과 숫자 간 연산 규칙

- DataFrame의 개별 원소의 숫자를 각각 연산
- 덧셈, 뺄셈, 곱셈, 나눗셈 모두 가능
- DataFrame 객체로 반환

(2) DataFrame와 숫자 간 연산 형식

- DataFrame 객체 + 연산자(사칙연산) + 숫자

1) DataFrame과 숫자 간 연산하기

(3) seaborn 패키지에서 제공하는 titanic 데이터셋에서 age를 읽어서 2로 나눔

- 함수형식
 - seaborn.load_dataset('데이터셋명')
→ 데이터셋을 읽어서 DataFrame을 생성
 - DataFrame객체.loc[추출할 행 범위, 열명]
→ 원하는 항목을 추출



```
import pandas as pd
import seaborn as sns
titan = sns.load_dataset("titanic")
df=titan.loc[10:16, ['age']]
res=df.div(2)
print(res)
```



	age
10	2.0
11	29.0
12	10.0
13	19.5
14	7.0
15	27.5
16	1.0

2) DataFrame 간 연산하기

(1) 크기가 같은 경우

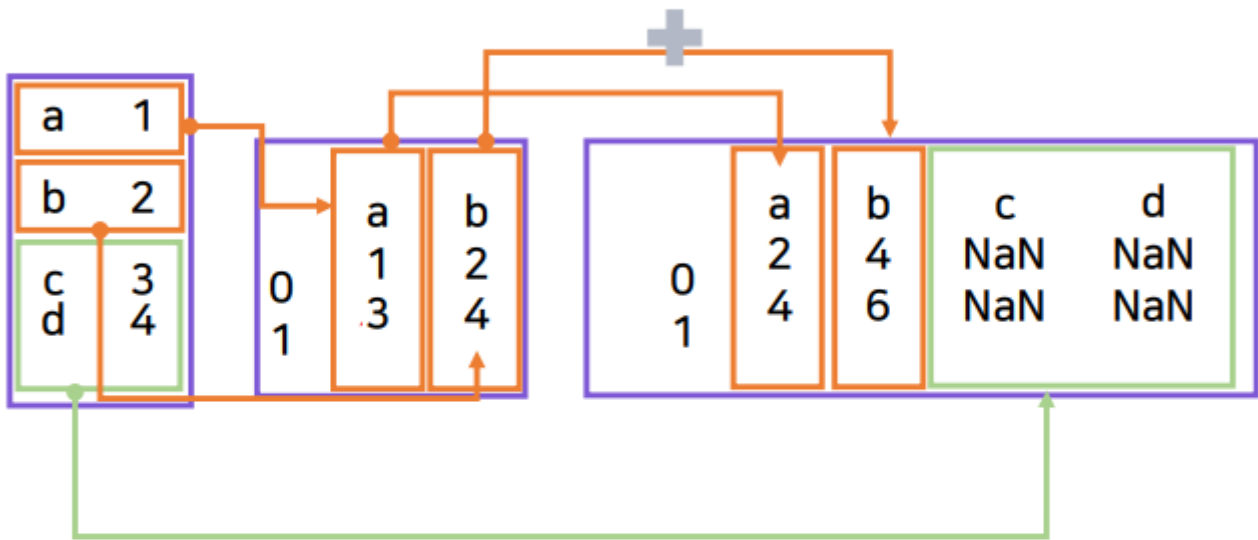
- 두 DataFrame 객체 간의 연산은 동일한 행 또는 열을 가진 요소끼리 연산 수행

(2) 크기가 다른 경우

- 두 DataFrame 객체 중 어느 한쪽에 요소가 존재하지 않거나 NaN이면 NaN으로 반환

3) Series와 DataFrame 간 연산하기

- (1) DataFrame의 행과 열을 통합 후 연산
- (2) Series의 index를 DataFrame의 column에 맞추고 연산
- (3) 짝이 안 맞으면 NaN 값을 반환



```

import pandas as pd
obj1 = pd.Series([1,2,3,4], index=['a','b','c','d'])
obj2=pd.DataFrame([[1,2],[3,4]], columns=['a','b'])
res=obj1.add(obj2)
print(res)
    
```

```

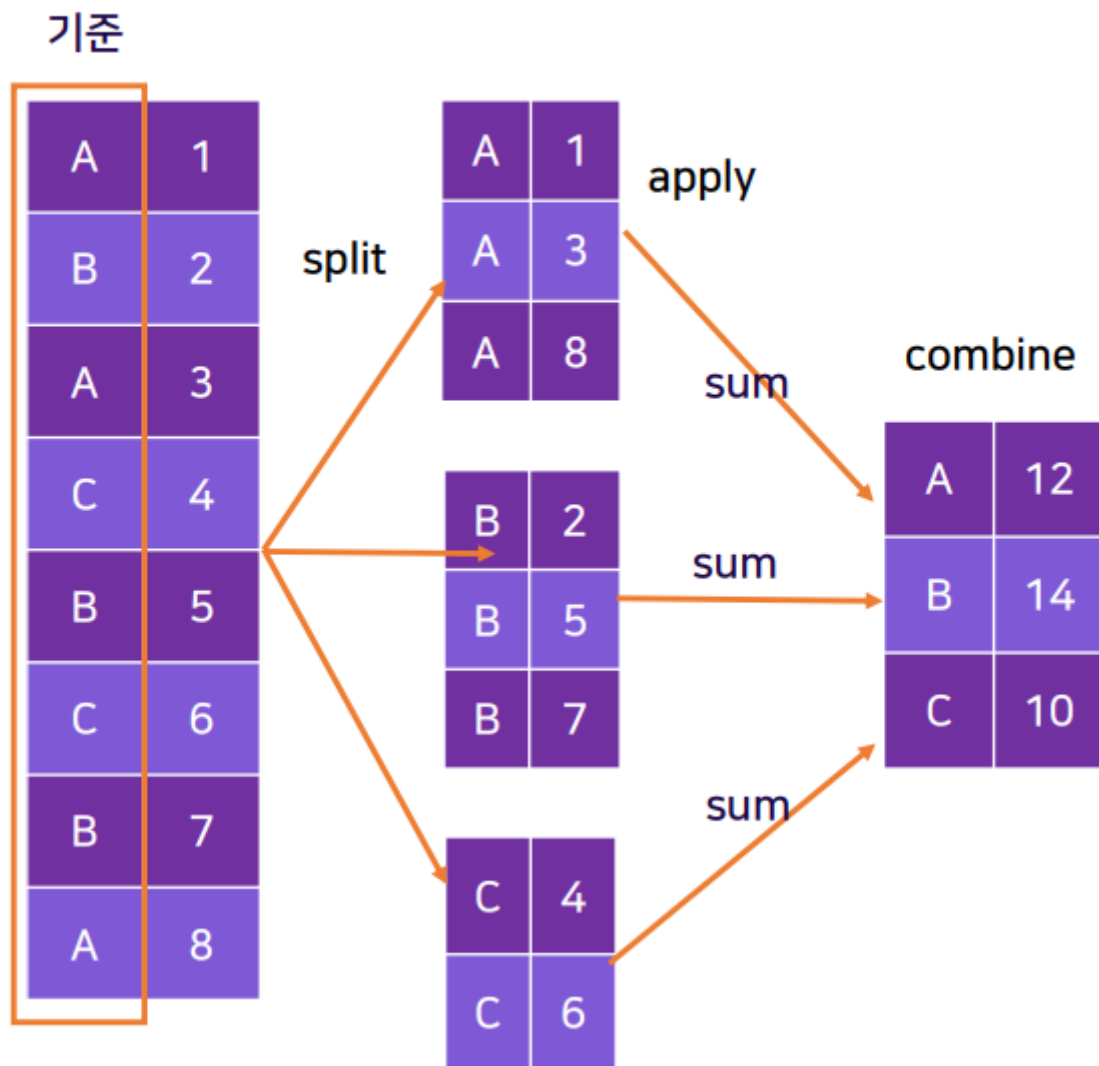
a  b  c  d
0  2  4 NaN NaN
1  4  6 NaN NaN
    
```

3. 그룹 객체 생성 및 연산하기

1) 그룹 연산 개요

- (1) 복잡한 데이터를 어떤 기준에 따라 연산
- (2) 특정 기준을 적용하여 그룹별로 연산
- (3) 데이터 집계, 변환, 필터링에 효율적
- (4) 그룹 연산 단계
 - ① 데이터를 특정 조건에 의해 분할 → split
 - ② 데이터를 집계, 변환, 필터링 → apply
 - ③ 2단계의 처리 결과를 하나로 결합 → combine

1) 그룹 연산 개요



2) 그룹 객체 생성하기

(1) groupby 함수 이용

(2) 특정 열을 기준으로 DataFrame을 분할하여 그룹 객체를 반환

- [함수형식] DataFrame 객체.groupby(그룹화할 열)
 - 그룹화할 열은 1개 이상 지정할 수 있음
- seaborn의 titanic 데이터셋에서 'survived', 'sex', 'age', 'embark_town'을 추출하여 생존여부 내에서 성별로 그룹화(분할)하기

```
import pandas as pd
import seaborn as sns
titan = sns.load_dataset("titanic")
df=titan.loc[7:16, ['survived', 'sex', 'age', 'embark_town']]
group_res=df.groupby(['survived', 'sex'])
print(group_res)
```



```
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x7fd4d2af8c10>
```

3) 그룹(분할) 객체 연산하기

(1) 데이터 집계

- 집계 함수를 이용하여 그룹별 통계를 낼 수 있음
- mean, max, min, sum, count, size, var 등
- 그룹화할 열을 제외한 모든 필드에 집계 함수 적용
- 특정 열에 집계 함수 적용도 가능
 - [함수형식] DataFrame 객체.groupby(그룹화할 열)[집계열].집계함수
 - 집계열은 생략할 수 있음
- seaborn의 titanic 데이터셋에서 'survived', 'sex', 'age', 'fare'를 추출하여 생존여부 내 성별로 나이와 요금의 평균을 구하기

```
import pandas as pd
import seaborn as sns
titan = sns.load_dataset("titanic")
df=titan.loc[7:16, ['survived', 'sex', 'age', 'fare']]
group_res=df.groupby(['survived', 'sex'])
res=group_res.mean()
print(res)
```



		age	fare
survived	sex		
0	female	14.00	7.85420
	male	15.75	22.38125
1	female	31.60	20.09082

3) 그룹(분할) 객체 연산하기

(1) 데이터 집계

- seaborn의 titanic 데이터셋에서 'survived', 'sex', 'age', 'fare'를 추출하여 생존여부 내에서 성별로 나이의 평균만 구하기

```
import pandas as pd
import seaborn as sns
titan = sns.load_dataset("titanic")
df=titan.loc[7:16, ['survived', 'sex', 'age', 'fare']]
group_res=df.groupby(['survived', 'sex'])
res=group_res.mean()['age']
print(res)
```



survived	sex	
0	female	14.00
	male	15.75
1	female	31.60

Name: age, dtype: float64

3) 그룹(분할) 객체 연산하기

(2) agg 함수

- 집계 연산을 포함한 사용자정의함수를 그룹 객체에 적용
 - 여러 개의 함수를 사용하여 데이터 집계 연산 처리 가능
 - 모든 열에 여러 개의 함수를 적용하거나, 열마다 서로 다른 함수를 적용하는 것도 가능
- 집계 연산을 포함한 사용자정의 함수 매핑
`group객체.agg(사용자정의 함수)`
 - 모든 열에 여러 함수 매핑
`group객체.agg([함수1, 함수2, ...])`
 - 열마다 서로 다른 함수 매핑
`group객체.agg({'열명1':함수1, '열명2':함수2, ...})`

3) 그룹(분할) 객체 연산하기

(2) agg 함수

- seaborn의 titanic 데이터셋에서 'survived', 'sex', 'age', 'fare'를 추출하여 생존 여부 내에서 성별로 평균-최소값을 구하기
- 이때, 평균에서 최소값을 빼는 사용자정의 함수를 이용

```
import pandas as pd
import seaborn as sns
def mean_min(x):
    return x.mean()-x.min()
titan = sns.load_dataset("titanic")
df=titan.loc[7:16, ['survived','sex','age','fare']]
group_res=df.groupby(['survived','sex'])
mean_min_res=group_res.agg(mean_min)
print(mean_min_res)
```

		age	fare
survived	sex		
0	female	0.00	0.00000
	male	13.75	14.33125
1	female	27.60	8.95752

3) 그룹(분할) 객체 연산하기

(2) agg 함수

- seaborn의 titanic 데이터셋에서 'survived', 'sex', 'age', 'fare'를 추출하여 생존 여부 내에서 성별로 평균-최솟값을 구하기

```
import pandas as pd
import seaborn as sns
titan = sns.load_dataset("titanic")
df=titan.loc[7:16, ['survived', 'sex', 'age', 'fare']]
group_res=df.groupby(['survived', 'sex'])
mean_min_res=group_res.agg(['mean', 'min'])
print(mean_min_res)
```

```

      age      fare
survived sex  mean  min  mean  min
0      female 14.00  14.0   7.85420  7.8542
      male   15.75   2.0  22.38125  8.0500
1      female 31.60   4.0  20.09082 11.1333
```

3) 그룹(분할) 객체 연산하기

(2) agg 함수

- seaborn의 titanic 데이터셋에서 'survived', 'sex', 'age', 'fare'를 추출하여 생존 여부 내에서 성별로 나이의 평균, 요금의 평균과 최솟값을 구하기

```
import pandas as pd
import seaborn as sns
titan = sns.load_dataset("titanic")
df=titan.loc[7:16, ['survived', 'sex', 'age', 'fare']]
group_res=df.groupby(['survived', 'sex'])
mean_min_res=group_res.agg({'age': 'mean', 'fare': ['mean', 'min']})
print(mean_min_res)
```

```
survived sex      age      fare      min
0      female  14.00    7.85420    7.8542
      male    15.75   22.38125    8.0500
1      female  31.60   20.09082   11.1333
```

6. Pandas 함수 정의 및 매핑

사용자 함수 정의 및 매핑하기

1. 사용자 함수 정의하기

1) 사용자 함수 개요

(1) 사용자 함수란?

- 사용자가 직접 만들어서 사용하는 함수

(2) 구조

- 함수 정의
 - `def 함수명([매개변수1[=기본값], 매개변수2, ...]):`
수행할 명령문1
[수행할 명령문1
...]
[return 반환값]
- 함수 호출
 - `함수명([전달 인자])`
- 용어
 - 매개변수: 함수에 입력으로 전달된 값을 받는 변수
 - 기본 매개변수: 전달된 입력값이 없으면 기본값을 받는 변수
 - 전달인자: 함수를 호출할 때 전달하는 입력값

2) 사용자 함수 정의하기

(1) 입력값(매개변수)과 반환값이 있는 경우

정의	<pre>def add(data1, data2): res=data1+data2 return res</pre>	<pre>def add(data1, data2): return data1+data2</pre>
호출	<pre>res=add(3,4) print(res)</pre>	<pre>print(add(3,4))</pre>

(2) 반환값만 있는 경우

정의	<pre>def Talk(): msg= 'Hi, Pandas!' return msg</pre>	<pre>def Talk(): return 'Hi, Pandas!'</pre>
호출	<pre>msg =Talk() print(msg)</pre>	<pre>print(Talk())</pre>

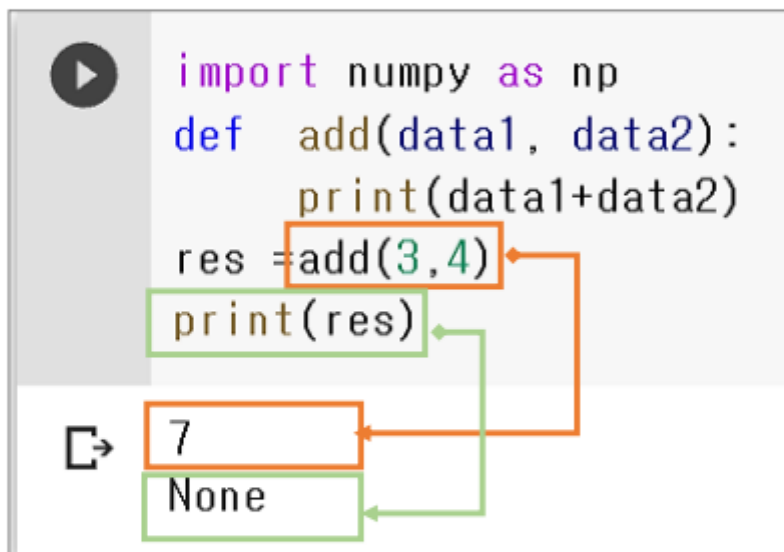
(3) 입력값(매개변수)만 있는 경우

정의	<pre>def add(data1, data2): print(data1+data2)</pre>
호출	<pre>add(3,4)</pre>

2) 사용자 함수 정의하기

(3) 입력값(매개변수)만 있는 경우

- 결과값을 반환하는 함수가 아니므로 함수 수행 결과를 변수에 저장할 수 없음
 - `res = add(3,4)`



(4) 입력값(매개변수)와 반환값이 모두 없는 경우

정의	<pre>def Talk(): print('Hi, Pandas!')</pre>
호출	<code>Talk()</code>

2) 사용자 함수 정의하기

(5) 입력값(매개변수)가 기본값을 갖는 경우

정의	<pre>def add(data1, data2=20): res=data1+data2 return res</pre>		
호출	<pre>res =add(3,5) print(res)</pre>	결과 →	8
	<pre>res =add(3) print(res)</pre>	결과 →	23

2. 함수 매핑하기

1) 함수 매핑이란?

(1) Series 또는 DataFrame의 개별 원소를 특정

- 함수에 일대일로 대응시키는 과정

(2) 사용자가 직접 만든 함수 매핑 가능

- Pandas 기본 함수로 처리하기 어려운 복잡한 연산 적용 가능

2) Series 원소에 함수 매핑하기

(1) 개별 원소 함수 매핑이란?

- `apply()` 함수 이용
- 인자로 전달하는 매핑 함수에 Series의 모든 원소를 하나씩 입력하고 함수의 반환값을 돌려받음
- Series 원소의 개수만큼 반환값을 받아서 같은 크기의 Series 객체로 반환함
 - `Series객체.apply(매핑 함수)`

(2) 개별 원소 함수 매핑 적용

- seaborn 패키지 내 데이터셋 중 "titanic"의 생존 여부(survived), 성별(sex), 나이(age), 요금(fare)을 불러와서 사용자 함수를 적용해봄

2) Series 원소에 함수 매핑하기

(2) 개별 원소 함수 매핑 적용

① 정의된 사용자 함수 매핑하기

```
# titanic 데이터셋을 읽기 위해서 seaborn를 import함
import seaborn as sns
# 사용자 함수를 만들
def minus(data1, data2):
    return data1-data2
# titanic 데이터셋을 읽어서 titanic 변수에 저장
titanic = sns.load_dataset('titanic')
# titanic 데이터셋에서 특정 속성만 추출하여 변수에 저장
df=titanic.loc[:,['survived','sex','age','fare']]
# 사용자 함수 minus를 매핑함
print(df['fare'].apply(minus, data2=10))
```

2) Series 원소에 함수 매핑하기

(2) 개별 원소 함수 매핑 적용

① 정의된 사용자 함수 매핑하기

▪ 코랩(Colab)에서 실행시키기

```
import seaborn as sns
def minus(data1, data2):
    return data1-data2
titanic = sns.load_dataset('titanic')
df=titanic.loc[:,['survived','sex','age','fare']]
print(df['fare'].apply(minus, data2=10))
```

```
0      -2.7500
1      61.2833
2      -2.0750
3      43.1000
4      -1.9500
...
886      3.0000
887     20.0000
888     13.4500
889     20.0000
890     -2.2500
Name: fare, Length: 891, dtype: float64
```

2) Series 원소에 함수 매핑하기

(2) 개별 원소 함수 매핑 적용

② lambda 함수 매핑하기

```
# titanic 데이터셋을 읽기 위해서 seaborn를 import함
import seaborn as sns

# 사용자 함수를 만듦
def minus(data1, data2):
    return data1-data2

# titanic 데이터셋을 읽어서 titanic 변수에 저장
titanic = sns.load_dataset('titanic')

# titanic 데이터셋에서 특정 속성만 추출하여 변수에 저장
df=titanic.loc[:,['survived','sex','age','fare']]

# lambda 함수를 이용하여 매핑함
print(df['fare'].apply(lambda x:minus(x, 10)))
```

2) Series 원소에 함수 매핑하기

(2) 개별 원소 함수 매핑 적용

② lambda 함수 매핑하기

▪ 코랩(Colab)에서 실행시키기

```
import seaborn as sns
def minus(data1, data2):
    return data1-data2
titanic = sns.load_dataset('titanic')
df=titanic.loc[:,['survived','sex','age','fare']]
print(df['fare']. apply(lambda x:minus(x, 10)))
```

```
0      -2.7500
1      61.2833
2      -2.0750
3      43.1000
4      -1.9500
...
886      3.0000
887     20.0000
888     13.4500
889     20.0000
890     -2.2500
Name: fare, Length: 891, dtype: float64
```

3) DataFrame 원소에 함수 매핑하기

(1) 개별 원소 함수 매핑이란?

- `applymap()` 함수 이용
- 인자로 전달하는 매핑 함수에 DataFrame의 모든 원소를 하나씩 입력하고 함수의 반환값을 돌려받음
- 동일한 형태의 DataFrame 객체로 반환함
 - `DataFrame객체.applymap(매핑 함수)`

(2) 개별 원소 함수 매핑 적용

- seaborn 패키지 내 데이터셋 중 "titanic"의 생존 여부(survived), 성별(sex), 나이(age), 요금(fare)을 불러와서 사용자 함수를 적용해봄

3) DataFrame 원소에 함수 매핑하기

(2) 개별 원소 함수 매핑 적용

① 정의된 사용자 함수 매핑하기

```
# titanic 데이터셋을 읽기 위해서 seaborn를 import함
import seaborn as sns

# 사용자 함수를 만듦
def minus(data):
    return data-10

# titanic 데이터셋을 읽어서 titanic 변수에 저장
titanic = sns.load_dataset('titanic')

# 모든 DataFrame 요소에 연산하므로 수치 데이터만 읽어옴
df=titanic.loc[:,[' age ', ' fare ' ]]

# 사용자 함수 minus를 매핑함
Print(df.applymap(minus))
```

3) DataFrame 원소에 함수 매핑하기

(2) 개별 원소 함수 매핑 적용

① 정의된 사용자 함수 매핑하기

▪ 코랩(Colab)에서 실행시키기

```
import seaborn as sns
def minus(data):
    return data-10
titanic = sns.load_dataset('titanic')
df=titanic.loc[:,['age', 'fare']]
print(df.applymap(minus))
```

```
↗
```

	age	fare
0	12.0	-2.7500
1	28.0	61.2833
2	16.0	-2.0750
3	25.0	43.1000
4	25.0	-1.9500
...	...	...
886	17.0	3.0000
887	9.0	20.0000
888	NaN	13.4500
889	16.0	20.0000
890	22.0	-2.2500

3) DataFrame 원소에 함수 매핑하기

(2) 개별 원소 함수 매핑 적용

② lambda 함수 매핑하기

```
# titanic 데이터셋을 읽기 위해서 seaborn를 import함
import seaborn as sns

# 사용자 함수를 만들
def minus(data):
    return data-10

# titanic 데이터셋을 읽어서 titanic 변수에 저장
titanic = sns.load_dataset('titanic')

# 모든 DataFrame 요소에 연산하므로 수치 데이터만 읽어옴
df=titanic.loc[:,[' age ', ' fare ' ]]

# lambda 함수를 이용하여 매핑함
print(df.applymap(lambda x:minus(x)))
```

3) DataFrame 원소에 함수 매핑하기

(2) 개별 원소 함수 매핑 적용

② lambda 함수 매핑하기

▪ 코랩(Colab)에서 실행시키기

```
import seaborn as sns
def minus(data):
    return data-10
titanic = sns.load_dataset('titanic')
df=titanic.loc[:,['age', 'fare']]
print(df.applymap(lambda x:minus(x)))
```

	age	fare
0	12.0	-2.7500
1	28.0	61.2833
2	16.0	-2.0750
3	25.0	43.1000
4	25.0	-1.9500
...	...	...
886	17.0	3.0000
887	9.0	20.0000
888	NaN	13.4500
889	16.0	20.0000
890	22.0	-2.2500

1) DataFrame의 각 열에 매핑하기

- (1) apply(axis=0) 함수 이용
- (2) 모든 열을 하나씩 분리하여 매핑 함수의 인자로 각 열(Serise)을 전달
- (3) Serise를 입력받고, Serise를 반환하는 함수를 매핑하면 DataFrame을 반환함
- (4) 매핑 함수에 따라 반환되는 객체의 종류가 다름
 - DataFrame객체.apply(매핑 함수, axis=0)
- (5) Serise를 입력받고 Serise를 반환하는 형태

```
# titanic 데이터셋을 읽기 위해서 seaborn를 import함
import seaborn as sns
# 각 요소가 null 인지를 판단하여 Serise 반환
def value_isnull(data):
    return data.isnull( )
# titanic 데이터셋을 읽어서 titanic 변수에 저장
titanic = sns.load_dataset('titanic')
# 모든 DataFrame 요소에 연산하므로 수치 데이터만 읽어옴
df=titanic.loc[:,['age', 'fare']]
# 사용자 함수를 매핑함
print(df.apply(value_isnull, axis=0))
```

1) DataFrame의 각 열에 매핑하기

(5) Serise를 입력받고 Serise를 반환하는 형태

- 코랩(Colab)에서 실행시키기

```
import seaborn as sns
def value_isnull(data):
    return data.isnull()
titanic = sns.load_dataset('titanic')
df=titanic.loc[:,['age','fare']]
print(df.apply(value_isnull, axis=0))
```

```
age  fare
0    False False
1    False False
2    False False
3    False False
4    False False
...    ...   ...
886  False False
887  False False
888   True  False
889  False False
890  False False
```

1) DataFrame의 각 열에 매핑하기

(6) Serise를 입력받고 하나의 값(통계값)으로 반환하는 형태

```
# titanic 데이터셋을 읽기 위해서 seaborn를 import함
import seaborn as sns

# 평균-최솟값
def mean_min(data):
    return data.mean( )-data.min( )

# titanic 데이터셋을 읽어서 titanic 변수에 저장
titanic = sns.load_dataset('titanic')

# 모든 DataFrame 요소에 연산하므로 수치 데이터만 읽어옴
df=titanic.loc[:,[ ' age ' , ' fare ' ]]

#사용자 함수를 매핑함
print(df.apply(mean_min))
```

1) DataFrame의 각 열에 매핑하기

(6) Serise를 입력받고 하나의 값(통계값)으로 반환하는 형태

- 코랩(Colab)에서 실행시키기

```
import seaborn as sns
def mean_min(data):
    return data.mean()-data.min()
titanic = sns.load_dataset('titanic')
df=titanic.loc[:,['age','fare']]
print(df.apply(mean_min))
```

```
age      29.279118
fare     32.204208
dtype: float64
```

2) DataFrame의 각 행에 매핑하기

- (1) apply(axis=1) 함수 이용
- (2) 모든 행을 하나씩 분리하여 매핑 함수의 인자로 각 행을 전달
- (3) DataFrame의 행 인덱스가 매핑 결과로 반환되는 Serise의 인덱스가 됨
- (4) 매핑 함수에 따라 반환되는 객체의 종류가 다름
 - DataFrame객체.apply(매핑 함수, axis=1)
- (5) 두 객체를 인자로 전달 받아 뺄셈(-)연산을 수행해봄

```
# titanic 데이터셋을 읽기 위해서 seaborn를 import함
import seaborn as sns
# 두 열의 차를 구하는 사용자 정의 함수
def minus(data1, data2):
    return data1-data2
# titanic 데이터셋을 읽어서 titanic 변수에 저장
titanic = sns.load_dataset('titanic')
# 모든 DataFrame 요소에 연산하므로 수치 데이터만 읽어옴
df=titanic.loc[:,['age','fare']]
# 사용자 정의 함수를 매핑함
print(df.apply(lambda x:minus(x['age'], x['fare']), axis=1))
```

2) DataFrame의 각 행에 매핑하기

(6) 코랩(Colab)에서 실행시켜 봄

```
▶ import seaborn as sns
def minus(data1, data2):
    return data1-data2
titanic = sns.load_dataset('titanic')
df=titanic.loc[:,['age','fare']]
print(df.apply(lambda x:minus(x['age'], x['fare']), axis=1))
```

```
↗ 0      14.7500
   1     -33.2833
   2      18.0750
   3     -18.1000
   4      26.9500
   ...
  886      14.0000
  887     -11.0000
  888         NaN
  889     -4.0000
  890     24.2500
Length: 891, dtype: float64
```

3) DataFrame 객체에 매핑하기

- (1) pipe() 함수 이용
- (2) 매핑 함수의 반환값에 따라 객체의 종류가 다름
- (3) DataFrame만 반환, Series만 반환, 개별 값 반환으로 나뉨
 - DataFrame객체.pipe(매핑 함수)

(4) DataFrame만 반환하는 형태

```
# titanic 데이터셋을 읽기 위해서 seaborn를 import함
import seaborn as sns
# DataFrame을 전달받아 null 여부를 판단한 DataFrame을 반환
def value_isnull(data):
    return data.isnull( )
# titanic 데이터셋을 읽어서 titanic 변수에 저장
titanic = sns.load_dataset('titanic')
# 모든 DataFrame 요소에 연산하므로 수치 데이터만 읽어옴
df=titanic.loc[:,['age','fare']]
# 사용자 함수를 매핑함
print(df.pipe(value_isnull))
```

3) DataFrame 객체에 매핑하기

(4) DataFrame만 반환하는 형태

- 코랩(Colab)에서 실행시켜 봄

```
import seaborn as sns
def value_isnull(data):
    return data.isnull()
titanic = sns.load_dataset('titanic')
df=titanic.loc[:,['age','fare']]
print(df.pipe(value_isnull))
```

```
➞
```

	age	fare
0	False	False
1	False	False
2	False	False
3	False	False
4	False	False
...	...	...
886	False	False
887	False	False
888	True	False
889	False	False
890	False	False

3) DataFrame 객체에 매핑하기

(5) Series만 반환하는 형태

```
# titanic 데이터셋을 읽기 위해서 seaborn를 import함
import seaborn as sns
# DataFrame의 각 열의 평균
def col_avg(data):
    return data.mean( )
# titanic 데이터셋을 읽어서 titanic 변수에 저장
titanic = sns.load_dataset('titanic')
# 모든 DataFrame 요소에 연산하므로 수치 데이터만 읽어옴
df=titanic.loc[:,['age','fare']]
# 사용자 함수를 매핑함
print(df.pipe(col_avg))
```

3) DataFrame 객체에 매핑하기

(6) 개별 값을 반환하는 형태

```
# titanic 데이터셋을 읽기 위해서 seaborn를 import함
import seaborn as sns
# DataFrame의 각 열의 개수
Def col_avg(data):
    return data.count( )
# DataFrame의 모든 데이터의 총계수
Def all_avg(data):
    return col_avg(data).sum( )
# titanic 데이터셋을 읽어서 titanic 변수에 저장
titanic = sns.load_dataset('titanic')
# 모든 DataFrame 요소에 연산하므로 수치 데이터만 읽어옴
df=titanic.loc[:,['age','fare']]
# 사용자 함수를 매핑함
print(df.pipe(all_avg))
```

3) DataFrame 객체에 매핑하기

(6) 개별 값을 반환하는 형태

- 코랩(Colab)에서 실행시켜 봄

```
import seaborn as sns
def col_avg(data):
    return data.count()
def all_avg(data):
    return col_avg(data).sum()
titanic = sns.load_dataset('titanic')
df=titanic.loc[:,['age', 'fare']]
print(df.pipe(all_avg))
```

1605

7. Pandas 데이터 전처리

1. 누락 데이터 처리하기

1) 누락 데이터 확인하기

(1) 누락 데이터 확인의 필요성

- 머신러닝 등 데이터 분석의 정확도
 - 데이터 품질에 좌우
- 데이터 품질 향상
 - 누락 데이터를 제거하거나 적절한 다른 값으로 대체
 - 중복 데이터 제거
 - 분석 목적에 맞게 변형
- 종종 원소 데이터 값이 누락되는 경우 발생
 - 입력 시 빠트리거나 변환 과정에서 소실

(2) 누락 데이터 확인

- 일반적으로 유효한 데이터 값이 존재하지 않는 누락 데이터를 NaN(Not a Number)로 표시함

	survived	pclass	sex	age	sibsp		deck	embark_town
0	0	3	male	22.0	1		NaN	Southampton
1	1	1	female	38.0	1		C	Cherbourg
2	1	3	female	26.0	0		NaN	Southampton
3	1	1	female	35.0	1		C	Southampton
4	0	3	male	35.0	0	...	NaN	Southampton
...	...	...	...	...	...	...	...	...
886	0	2	male	27.0	0		NaN	Southampton
887	1	1	female	19.0	0		B	Southampton
888	0	3	female	NaN	1		NaN	Southampton
889	1	1	male	26.0	0		C	Cherbourg
890	0	3	male	32.0	0		NaN	Queenstown
891 rows x 15 columns								

(2) 누락 데이터 확인

- info() 함수를 이용하여 요약 정보를 출력해 보면, 유효한 데이터 수를 보여 줌

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column          Non-Null Count  Dtype
---  -
0   survived        891 non-null    int64
1   pclass          891 non-null    int64
2   sex             891 non-null    object
3   age             714 non-null    float64
4   sibsp           891 non-null    int64
5   parch           891 non-null    int64
6   fare            891 non-null    float64
7   embarked        889 non-null    object
8   class           891 non-null    category
9   who             891 non-null    object
10  adult_male      891 non-null    bool
11  deck            203 non-null    category
12  embark_town     889 non-null    object
13  alive           891 non-null    object
14  alone           891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.6+ KB
```

(2) 누락 데이터 확인

- seaborn 라이브러리의 titanic 데이터셋을 활용하여
누락 데이터를 확인하기

① 실습을 위한 데이터셋을 가지고 있는 라이브러리를 import

- `import seaborn as sns`

② seaborn 라이브러리에서 titanic 데이터셋을 읽어서 DataFrame에 저장

- `df=sns.load_dataset('titanic')`

③ `info()` 함수를 이용하여 titanic 데이터셋의 누락 정도를 확인

- `df.info()`
 - 총 데이터수 : 891 → Range index
 - age의 누락 데이터 수 = $891 - 714 = 177$
 - embarked의 누락 데이터 수 = $891 - 889 = 2$
 - embark_town의 누락 데이터 = $891 - 889 = 2$
 - deck의 누락 데이터 수 = $891 - 203 = 688$

③ info() 함수를 이용하여 titanic 데이터셋의 누락 정도를 확인

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 891 entries, 0 to 890  
Data columns (total 15 columns):  
#   Column                Non-Null Count  Dtype    
---  ---                  
0   survived              891 non-null    int64    
1   pclass                 891 non-null    int64    
2   sex                    891 non-null    object    
3   age                    714 non-null    float64   
4   sibsp                  891 non-null    int64    
5   parch                  891 non-null    int64    
6   fare                   891 non-null    float64   
7   embarked               889 non-null    object    
8   class                  891 non-null    category   
9   who                    891 non-null    object    
10  adult_male             891 non-null    bool      
11  deck                   203 non-null    category   
12  embark_town            889 non-null    object    
13  alive                  891 non-null    object    
14  alone                  891 non-null    bool      
dtypes: bool(2), category(2), float64(2), int64(8)  
memory usage: 80.6+ KB
```

- ④ 유일한 값별 개수를 구하는 `value_counts()` 함수를 이용하여
누락 데이터의 개수를 확인하기

[함수 형식]

`value_counts`

(

`normalize=False,`

False면 개수, True면 상대 비율 구함

`sort=True,`

True면 개수 기준 정렬, False면 유일한 값 기준 정렬

`ascending=False,`

False면 내림차순 정렬, True면 오름차순 정렬

`bins=None,`

None이면 유일값 기준 개수, None 아니면 Bins Group별 개수

`dropna=True`

True면 NaN 무시, False면 유일값에 NaN 포함

)

(2) 누락 데이터 확인

- NaN의 개수로 구해야 하므로 dropna 속성에 False값을 할당함

```
df['필드명'].value_counts(dropna=False)
```

- deck 필드의 유일한 값별 개수를 구하기

```
df['deck'].value_counts(dropna=False)
```

```
NaN      688
C         59
B         47
D         33
E         32
A         15
F         13
G          4
Name: deck, dtype: int64
```

- NaN의 개수로 구해야 하므로 dropna 속성에 False값을 할당함

```
df['필드명'].value_counts(dropna=False)
```

- age 필드의 유일한 값별 개수를 구하기

```
df['age'].value_counts(dropna=False)
```

```
NaN      177
24.00     30
22.00     27
18.00     26
28.00     25
...
36.50      1
55.50      1
66.00      1
23.50      1
0.42       1
Name: age, Length: 89, dtype: int64
```

▶ 여기서 잠깐

(1) 누락 데이터를 직접 찾는 방법

- `isnull( )` 또는 `notnull( )` 함수와 `sum` 함수를 융합하여 구함
 - `isnull` → 누락 데이터이면 `True`, 그렇지 않으면 `False`
 - `notnull` → 유효한 값이면 `True`, 누락 데이터이면 `False`
 - `True`는 1, `False`는 0으로 환산되므로 `sum(axis=0)` 함수와 융합하면 누락 데이터의 개수를 구할 수 있음

(2) 누락 데이터 확인

- ⑤ isnull 함수와 sum 함수를 융합하여 누락 데이터의 개수를 확인하기

- `df.isnull().sum(axis=0)`

survived	0
pclass	0
sex	0
age	177
sibsp	0
parch	0
fare	0
embarked	2
class	0
who	0
adult_male	0
deck	688
embark_town	2
alive	0
alone	0
dtype:	int64

2) 누락 데이터 제거하기

(1) 누락 데이터 현황 분석

- 누락 데이터 확인
 - `df.isnull().sum(axis=0)`

age	177	deck	688
sibsp	0	embark_town	2
parch	0	alive	0
fare	0	alone	0
embarked	2	dtype: int64	

- deck의 경우 너무 많은 데이터가 누락된 상태
 - 필드 자체를 삭제하는 것이 적합

2) 누락 데이터 제거하기

(2) dropna 함수

- NaN 또는 np.nan이 포함된 행 또는 열 삭제

[함수 형식]

DataFrame객체.dropna(subset, how, axis, thresh)

- subset: 삭제하고자 하는 행 또는 열 이름이 있는 배열
 - how: 행 또는 열을 삭제하는 방법
 - any: NaN 또는 np.nan이 들어 있는 행 또는 열 삭제
 - all: 모든 값이 NaN 또는 np.nan인 행 또는 열 삭제
 - axis: 행(0)을 삭제할지, 열(1)을 삭제할지를 결정
 - thresh: 행이랑 열을 삭제하기 위한 최소 비결측 값
- DataFrame으로 반환

(3) 아래의 DataFrame의 누락 데이터를 제거하기

	Name	Gender	Age
0	Tom	male	NaN
1	Laura	None	25.0
2	Kevin	male	30.0
3	Julia	None	18.0
4	Jhon	male	NaN

- 위의 DataFrame을 생성하기

```
import pandas as pd
df=pd.DataFrame({
    'Name':{0:'Tom',1:'Laura',2:'Kevin',3:'Julia',4:'Jhon'},
    'Gender':{0:'male',1:None,2:'male',3:None,4:'male'},
    'Age':{0:None,1:25,2:30,3:18,4:None}})
```

(3) 아래의 DataFrame의 누락 데이터를 제거하기

- NaN이 포함된 모든 행 또는 열을 삭제하기

	Name	Gender	Age
0	Tom	male	NaN
1	Laura	None	25.0
2	Kevin	male	30.0
3	Julia	None	18.0
4	Jhon	male	NaN



```
import pandas as pd
df=pd.DataFrame({
    'Name': {0: 'Tom', 1: 'Laura', 2: 'Kevi
    'Gender': {0: 'male', 1: None, 2: 'male'
    'Age': {0: None, 1: 25, 2: 30, 3: 18, 4:
    })
print(df.dropna())
```

	Name	Gender	Age
2	Kevin	male	30.0

(3) 아래의 DataFrame의 누락 데이터를 제거하기

- NaN이 포함된 모든 열을 삭제하기

	Name	Gender	Age
0	Tom	male	NaN
1	Laura	None	25.0
2	Kevin	male	30.0
3	Julia	None	18.0
4	Jhon	male	NaN



```
import pandas as pd
df=pd.DataFrame({
    'Name': {0: 'Tom', 1: 'Laura', 2: 'Kevin',
    'Gender': {0: 'male', 1: None, 2: 'male',
    'Age': {0: None, 1: 25, 2: 30, 3: 18, 4:
    })
print(df.dropna(axis=1))
```



```
      Name
0      Tom
1  Laura
2  Kevin
3  Julia
4   Jhon
```

(3) 아래의 DataFrame의 누락 데이터를 제거하기

- "Age" 필드의 NaN이 포함된 행을 삭제하기

	Name	Gender	Age
0	Tom	male	NaN
1	Laura	None	25.0
2	Kevin	male	30.0
3	Julia	None	18.0
4	Jhon	male	NaN

```
import pandas as pd
df=pd.DataFrame({
    'Name':{0:'Tom',1:'Laura',2:'Kevin',3:'Julia',4:'Jhon'},
    'Gender':{0:'male',1:None,2:'male',3:None,4:'male'},
    'Age':{0:None,1:25,2:30,3:18,4:None}})
print(df.dropna(subset=['Age'],how='any',axis=0))
```

```
➡
```

	Name	Gender	Age
1	Laura	None	25.0
2	Kevin	male	30.0
3	Julia	None	18.0

(3) 아래의 DataFrame의 누락 데이터를 제거하기

- titanic 데이터셋에서 "deck" 열을 삭제하기
 - 열을 삭제하여야 하므로 axis를 1로, deck는 688명이 누락되었으므로, thresh값을 500으로 설정



```
import seaborn as sns
df=sns.load_dataset('titanic')
df.dropna(axis=1, thresh=500)
```



	survived	pclass	sex	age	sibsp	parch	fare	embarked
0	0	3	male	22.0	1	0	7.2500	S
1	1	1	female	38.0	1	0	71.2833	C
2	1	3	female	26.0	0	0	7.9250	S
3	1	1	female	35.0	1	0	53.1000	S
4	0	3	male	35.0	0	0	8.0500	S
...	...	...	...	...	...	...	...	...
886	0	2	male	27.0	0	0	13.0000	S
887	1	1	female	19.0	0	0	30.0000	S
888	0	3	female	NaN	1	2	23.4500	S
889	1	1	male	26.0	0	0	30.0000	C
890	0	3	male	32.0	0	0	7.7500	Q

891 rows × 14 columns

3) 누락 데이터 치환하기

(1) fillna 함수

- NaN 또는 np.nan을 다른 값으로 대체

[함수 형식]

DataFrame객체.fillna(value, method, axis, inplace)

- value: 대체할 값(scalar, dic, series, dataframe)
- method: NaN값을 채우는 방법(backfill, bfill, pad, ffill)
- axis: 행(0) 방향으로 채울지, 열(1) 방향으로 채울지를 결정
- inplace: 원본을 수정할지를 결정(True/False)

(2) titanic "age"열의 누락 데이터를 평균으로 치환

- age열의 누락데이터는 삭제보다는 평균으로 치환하는 것이 적합
- mean 함수를 이용하여 age열의 NaN을 제외한 값들의 평균을 먼저 구함
 - age_mean=df['age'].mean(axis=0)
- fillna 함수를 이용하여 age 데이터의 평균으로 치환
 - df['age'].fillna(age_mean, inplace=True)

3) 누락 데이터 치환하기

(3) 실습하기

```
import seaborn as sns
df=sns.load_dataset('titanic')
age_mean=df['age'].mean(axis=0)
df['age'].fillna(age_mean, inplace=True)
df
```

	survived	pclass	sex	age
0	0	3	male	22.0
1	1	1	female	38.0
2	1	3	female	26.0
3	1	1	female	35.0
4	0	3	male	35.0
...	...	...	...	...
886	0	2	male	27.0
887	1	1	female	19.0
888	0	3	female	NaN
889	1	1	male	26.0
890	0	3	male	32.0

pclass	sex	age	sibs
3	male	22.000000	
1	female	38.000000	
3	female	26.000000	
1	female	35.000000	
3	male	35.000000	
...	...	...	...
2	male	27.000000	
1	female	19.000000	
3	female	29.699118	
1	male	26.000000	
3	male	32.000000	

3) 누락 데이터 치환하기

(4) titanic "embark_town" 열의 누락 데이터를 이웃데이터로 치환

- 승선 도시가 저장된 embark_town은 이웃하는 데이터끼리 유사성을 가질 확률이 높음

 <pre>import seaborn as sns df=sns.load_dataset('titanic') print(df[825:835])</pre>										
										
	survived	pclass	sex	age	...	deck	embark_town	alive	alone	
825	0	3	male	NaN	...	NaN	Queenstown	no	True	
826	0	3	male	NaN	...	NaN	Southampton	no	True	
827	1	2	male	1.00	...	NaN	Cherbourg	yes	False	
828	1	3	male	NaN	...	NaN	Queenstown	yes	True	
829	1	1	female	62.00	...	B	NaN	yes	True	
830	1	3	female	15.00	...	NaN	Cherbourg	yes	False	
831	1	2	male	0.83	...	NaN	Southampton	yes	False	
832	0	3	male	NaN	...	NaN	Cherbourg	no	True	
833	0	3	male	23.00	...	NaN	Southampton	no	True	
834	0	3	male	18.00	...	NaN	Southampton	no	True	

3) 누락 데이터 치환하기

(5) 829행의 다음 행의 값으로 채우는 방법

- `df['embark_town'].fillna(method='bfill', inplace=True)`

```
born as sns
d_dataset('titanic')
_ town'].fillna(method='bfill', inplace=True)]
25:835])
```

ved	pclass	sex	age	...	deck	embark_town	alive
0	3	male	NaN	...	NaN	Queenstown	no
0	3	male	NaN	...	NaN	Southampton	no
1	2	male	1.00	...	NaN	Cherbourg	yes
1	3	male	NaN	...	NaN	Queenstown	yes
1	1	female	62.00	...	B	Cherbourg	yes
1	3	female	15.00	...	NaN	Cherbourg	yes
1	2	male	0.83	...	NaN	Southampton	yes
0	3	male	NaN	...	NaN	Cherbourg	no
0	3	male	23.00	...	NaN	Southampton	no
0	3	male	18.00	...	NaN	Southampton	no

3) 누락 데이터 치환하기

(6) 829행의 이전 행의 값으로 채우는 방법

- `df['embark_town'].fillna(method='ffill', inplace=True)`

```

# Load data as sns
sns.load_dataset('titanic')
df['embark_town'].fillna(method='ffill', inplace=True)
df[825:835]

```

survived	pclass	sex	age	...	deck	embark_town	alive
0	3	male	NaN	...	NaN	Queenstown	no
0	3	male	NaN	...	NaN	Southampton	no
1	2	male	1.00	...	NaN	Cherbourg	yes
1	3	male	NaN	...	NaN	Queenstown	yes
1	1	female	62.00	...	B	Queenstown	yes
1	3	female	15.00	...	NaN	Cherbourg	yes
1	2	male	0.83	...	NaN	Southampton	yes
0	3	male	NaN	...	NaN	Cherbourg	no
0	3	male	23.00	...	NaN	Southampton	no
0	3	male	18.00	...	NaN	Southampton	no

2. 중복 데이터 처리하기

1) 중복 데이터 확인하기

(1) duplicated 함수

- 중복된 행이 있는지를 점검
- 이전 행과 중복되면 **True**, 그렇지 않으면 **False**

[함수 정의]

`DataFrame객체.duplicated(field, keep)`

- field: 중복을 검사할 열명
- keep: 중복값 표시 방법(first, last, False)
 - first: 첫 번째 값은 False, 두 번째 값부터는 True
 - last: 첫 번째 값은 True, 두 번째 값부터는 False
 - False: 중복이 발견되면 모든 행을 True로 표기

1) 중복 데이터 확인하기

(2) duplicated를 이용한 중복 데이터 확인

```
import pandas as pd
df=pd.DataFrame({
    'User_Code':{0:'T1001',1:'S1000',2:'T1001',3:'S1000',4:'K1000'},
    'Gender':{0:'male',1:None,2:'male',3:None,4:'male'},
    'Age':{0:None,1:25,2:30,3:18,4:None}})
print(df.duplicated(['User_Code']))
```

```
0    False
1    False
2     True
3     True
4    False
dtype: bool
```



	User_Code	Gender	Age
0	T1001	male	NaN
1	S1000	None	25.0
2	T1001	male	30.0
3	S1000	None	18.0
4	K1000	male	NaN

2) 중복 데이터 제거하기

(1) drop_duplicates 함수

- 중복된 데이터를 삭제

[함수 정의]

DataFrame객체.drop_duplicates(subset)

- subset: 중복을 검사할 열

(2) drop_duplicates를 이용한 중복 데이터 제거

- 첫 번째 데이터를 제외한 모든 중복 데이터를 삭제하기

```
import pandas as pd
df=pd.DataFrame({
    'User_Code':{0:'T1001',1:'S1000',2:'T1001',3:'S1000',4:'K1000'},
    'Gender':{0:'male',1:None,2:'male',3:None,4:'male'},
    'Age':{0:None,1:25,2:30,3:18,4:None}})
print(df.drop_duplicates(subset=['User_Code']))
```

	User_Code	Gender	Age
0	T1001	male	NaN
1	S1000	None	25.0
4	K1000	male	NaN

	User_Code	Gender	Age
0	T1001	male	NaN
1	S1000	None	25.0
2	T1001	male	30.0
3	S1000	None	18.0
4	K1000	male	NaN

3. Series 데이터 전처리하기

1) 수집 데이터를 Series 객체로 변환하기

(1) to_datetime 함수

- 날짜 형식의 문자열을 datetime으로 변환

[함수 정의]

pandas.to_datetime(param, format)

- param: 변환할 열명
- format: 날짜 변환 형식 문자열

%Y	%m	%d	%H	%M	%S
년	월	일	시	분	초

1) 수집 데이터를 Series 객체로 변환하기

(2) to_datetime 실습하기

```
import pandas as pd
df=pd.DataFrame({
    'Name':{0:'Tom',1:'Laura',2:'Kevin',3:'Julia',4:'Jhon'},
    'Birthday':{0:'1997-10-10',1:'2002-04-18',2:'1981-11-12',
                3:'2000-12-31',4:'1999-04-09'}})
df['New_Birthday']=pd.to_datetime(df['Birthday'],
                                   format="%Y-%m-%d")
print(df.info())
```

#	Column	Non-Null Count	Dtype
0	Name	5 non-null	object
1	Birthday	5 non-null	object
2	New_Birthday	5 non-null	datetime64[ns]

dtypes: datetime64[ns](1), object(2)
memory usage: 160.0+ bytes

2) Series 데이터 만들기

(1) date_range 함수

- 일정 간격으로 시간 데이터를 생성

[함수 정의]

`pandas.date_range(start, end, periods, freq, tz)`

- start, end: 데이터 시작, 끝 시간 지정
- periods: 기간의 수를 지정
- freq: 시간 간격 설정

D	W	MS	M	H	T	S
일별	주별	월별 시작일	월별 말일	시간별	분별	초별

- tz: 시간대(time zone) 설정

2) Series 데이터 만들기

(2) date_range를 이용하여 Series 생성하기

- 2019년 1월 1일부터 12월 31일까지 3일 간격으로 날짜 데이터를 생성하기

- import pandas as pd

```
dates=pd.date_range(start='2019-01-01', end='2019-12-31', freq='3D')
```

```
print(dates)
```

```
DatetimeIndex(['2019-01-01', '2019-01-04', '2019-01-07', '2019-01-10', '2019-01-13', '2019-01-16', '2019-01-19', '2019-01-22', '2019-01-25', '2019-01-28', ..., '2019-12-03', '2019-12-06', '2019-12-09', '2019-12-12', '2019-12-15', '2019-12-18', '2019-12-21', '2019-12-24', '2019-12-27', '2019-12-30'], dtype='datetime64[ns]', length=122, freq='3D')
```

2) Series 데이터 만들기

(3) period_range 함수

- 일정 간격으로 시간 데이터를 생성

[함수 정의]

pandas.period_range(start, end, periods, freq)

- start: 데이터 시작 지정
- end: 끝 시간 지정(생략 가능)
- periods: 기간의 수를 지정
- freq: 시간 간격 설정

D	W	MS	M	H	T	S
일별	주별	월별 시작일	월별 말일	시간별	분별	초별

2) Series 데이터 만들기

(4) period_range를 이용하여 Series 생성하기

- 2019년 1월 1일부터 1월 31일까지 3시간 간격으로 날짜 데이터를 생성하기

- import pandas as pd

```
date_times=pd.period_range(start='2019-01-01',  
                             periods=20, freq='3H')  
print(date_times)
```

```
PeriodIndex(['2019-01-01 00:00', '2019-01-01 03:00', '2019-01-01 06:00',  
            '2019-01-01 09:00', '2019-01-01 12:00', '2019-01-01 15:00',  
            '2019-01-01 18:00', '2019-01-01 21:00', '2019-01-02 00:00',  
            '2019-01-02 03:00', '2019-01-02 06:00', '2019-01-02 09:00',  
            '2019-01-02 12:00', '2019-01-02 15:00', '2019-01-02 18:00',  
            '2019-01-02 21:00', '2019-01-03 00:00', '2019-01-03 03:00',  
            '2019-01-03 06:00', '2019-01-03 09:00'],  
            dtype='period[3H]', freq='3H')
```

8. Pandas 데이터 입출력 및 분석

데이터 입출력하기

1. 외부 데이터 읽기

1) CSV 파일 읽기

(1) read_csv 함수

- csv 파일은 각 항목을 콤마(,)로 구분된 것을 의미

[함수 형식]

```
pandas.read_csv(filepath, header, index_col, names)
```

- filepath: 가져올 csv 파일의 위치
- header: 헤더의 열이름으로 사용할 행 또는 열 지정
- index_col: DataFrame의 행 인덱스가 되는 열 지정
- names: 열이름으로 지정할 문자열

(2) read_csv 함수 활용

- user_info.csv 파일의 no 필드를 행 인덱스로 지정하여 읽어 보기

```
import pandas as pd  
df=pd.read_csv("user_info.csv", index_col='no')  
print(df)
```

	no	name	tel	age
1	1	홍길동	010-4111-7849	40
2	2	최영	010-4231-7849	35
3	3	이기자	010-3411-7849	50
4	4	송수현	010-231-7849	380
5	5	정명길	010-1111-7849	20

2) Excel 파일 읽기

(1) read_excel 함수

- 엑셀파일을 읽어서 DataFrame 형태로 반환해 줌

[함수 정의]

```
pandas.read_excel(filepath, header, index_col, names ,  
sheet_name)
```

- filepath: 가져올 csv 파일의 위치
- header: 헤더의 열이름으로 사용할 행 또는 열 지정
- index_col: DataFrame의 행 인덱스가 되는 열 지정
- names: 열이름으로 지정할 문자열
- sheet_name: 기본값은 0, 읽고자 하는 시트명 또는 인덱스

2. 내장 그래프 도구 활용하기

1) Matplotlib 활용하기

- (1) 직선 또는 곡선으로 연결하여 데이터 값 사이의 관계를 표시
- (2) 연속적인 값의 변화와 패턴을 파악하는 데 적합
- (3) pyplot 서브패키지를 이용하여 그래프를 만들고 변화를 줌
- (4) subplot 함수를 이용하면, Multi Graph를 그릴 수 있음
- (5) show 함수를 이용하여 그래프를 화면에 나타냄
- (6) savefig 함수를 이용하여 파일로 저장
- (7) 선 그래프 그리기
 - pyplot의 plot 함수 이용

[함수 형식]

```
pyplot.plot(x축값, y축값, color, linestyle, marker,  
markerfacecolor, markersize, linewidth)
```

- color: 선의 색(HTML color name 또는 Hex code)
- linestyle: 선 종류로 사용할 문자
- marker: 표식으로 사용할 문자
- markerfacecolor: 표식의 색 지정
- markersize: 표식의 크기
- linewidth: 선 굵기

(8) HTML color name

- https://www.w3schools.com/colors/colors_names.asp

(9) Hex code

- https://www.w3schools.com/colors/colors_picker.asp

AliceBlue #F0F8FF	AntiqueWhite #FAEBD7	Aqua #00FFFF
Aquamarine #7FFFD4	Azure #F0FFFF	Beige #F5F5DC
Bisque #FFE4C4	Black #000000	BlanchedAlmond #FFEBCD
Blue #0000FF	BlueViolet #8A2BE2	Brown #A52A2A

(10) 차트의 제목 표시 → title(제목)

(11) 축 이름 표시 → xlabel(x축 이름), ylabel(y축 이름)

(12) 항목 레이블 표시 → legend(labels, loc, fontsize)

- labels: 항목 레이블 목록
- loc: 항목 표시 위치 설정

0	1	2	3	4	5
best	upper right	upper left	lower left	lower right	right

6	7	8	9	10
center left	center right	lower center	lower center	center

- fontsize: 레이블 글자 크기 지정

(13) X축 눈금 라벨 공간 → figure(figsize(가로, 세로))

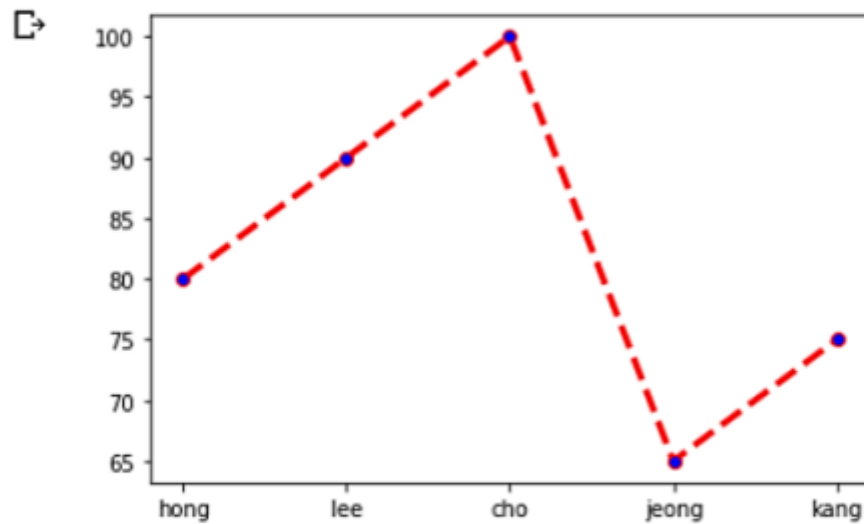
(14) 축 눈금 라벨 회전 → yticks, xticks

- rotation: 눈금 레벨 회전(vertical,숫자)
- size: 글자 크기 지정

(15) Y축의 범위 지정 → ylim(최솟값, 최댓값)

(16) 선 그래프 그리기

```
▶ from matplotlib import pyplot as plt  
plt.plot(['hong', 'lee', 'cho', 'jeong', 'kang'], [80, 90, 100, 65, 75],  
         color='red', linestyle='--', linewidth=3, marker='o',  
         markerfacecolor='blue')  
plt.show()
```



(17) 막대 그래프 그리기

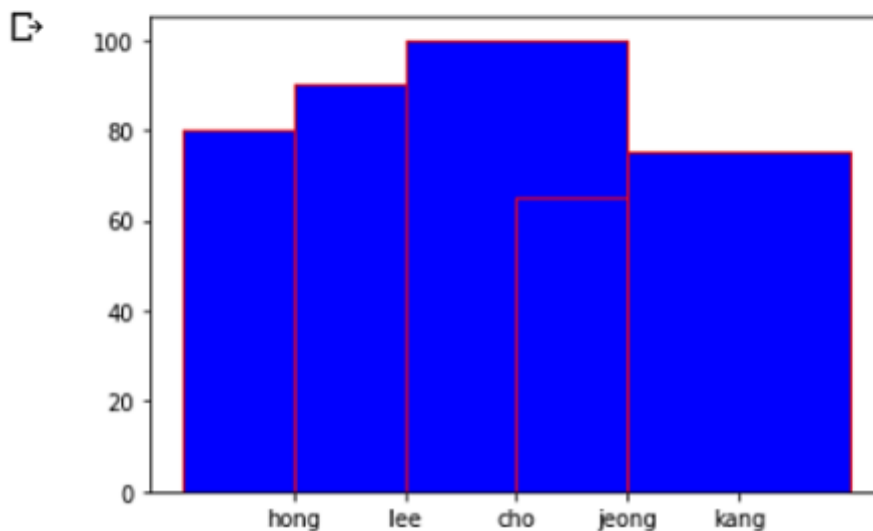
- pyplot의 bar 함수 이용

[함수 정의]

`pyplot.bar(x축값, y축값, color, edgecolor, width, alpha)`

- color:면의 색
- edgecolor: 면의 테두리 색
- width: 막대 폭 지정
- alpha: 투명도 지정

```
▶ from matplotlib import pyplot as plt  
plt.bar(['hong', 'lee', 'cho', 'jeong', 'kang'], [80, 90, 100, 65, 75],  
        color='blue', edgecolor='red', width=2)  
plt.show()
```



(18) 히스토그램 그래프 그리기

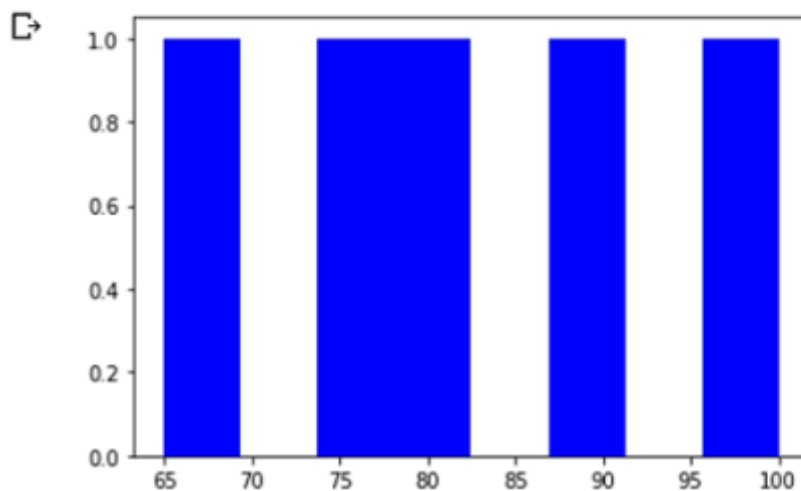
- pyplot의 hist 함수 이용

[함수 정의]

pyplot.hist(x, bins, histtype, facecolor, alpha)

- x: 그래프를 그릴 데이터
- bins: 히스토그램의 가로축 구간의 개수
- histtype: 히스토그램의 종류(bar, barstacked, step, stepfilled)
- facecolor: 히스토그램의 색
- alpha: 투명도 지정

```
from matplotlib import pyplot as plt
plt.hist([80, 90, 100, 65, 75],
         bins=8, color='red', histtype='bar', facecolor='blue')
plt.show()
```



(19) 선점도 그래프 그리기

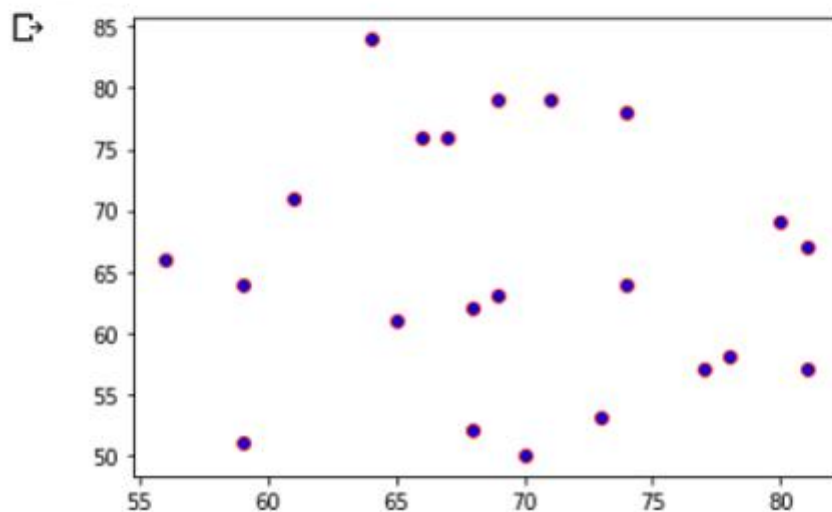
- pyplot의 scatter 함수 이용

[함수 정의]

`pyplot.scatter(x축값, y축값, s, c, edgecolor, alpha)`

- s: 마커의 크기 지정
- c: 마커의 색상 지정
- edgecolor: 선점도의 테두리 색
- alpha: 투명도 지정

```
import matplotlib.pyplot as plt
weight = [68, 81, 64, 56, 78, 74, 61, 77, 66, 68, 59, 71,
          80, 59, 67, 81, 69, 73, 69, 74, 70, 65]
weight2 = [52, 67, 84, 66, 58, 78, 71, 57, 76, 62, 51, 79,
           69, 64, 76, 57, 63, 53, 79, 64, 50, 61]
plt.scatter(weight, weight2, c='blue', edgecolor='red')
plt.show()
```



2) seaborn 활용하기

- (1) Matplotlib의 기능과 스타일을 확장한 시각화 도구
- (2) seaborn를 설치해야 함(Anaconda는 기본 설치)
- (3) 내포 데이터셋

'anagrams', 'anscombe', 'attention', 'brain_networks',
'car_crashes', 'diamonds', 'dots', 'exercise', 'flights', 'fmri',
'gammas', 'geyser', 'iris', 'mpg', 'penguins', 'planets', 'tips',
'titanic'

(4) 선 그래프 그리기

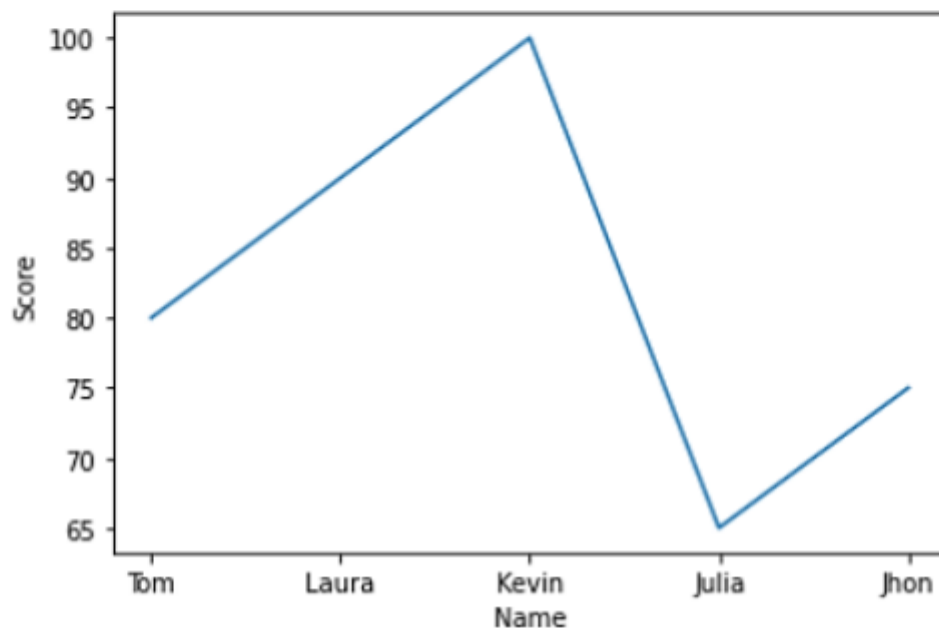
[함수 정의]

`seaborn.lineplot(x, y, data)`

- x: x축 레이블
- y: y축 레이블
- data: 그래프 그릴 데이터

```
import pandas as pd
import seaborn as sns
df=pd.DataFrame({
    'Name':{0:'Tom',1:'Laura',2:'Kevin',3:'Julia',4:'Jhon'},
    'Score':{0:80,1:90,2:100,3:65,4:75}})
sns.lineplot(x='Name', y='Score', data=df)
```

➞ <matplotlib.axes._subplots.AxesSubplot at 0x7f9b822aab50>



(5) 막대 그래프 그리기

- barplot 함수 이용

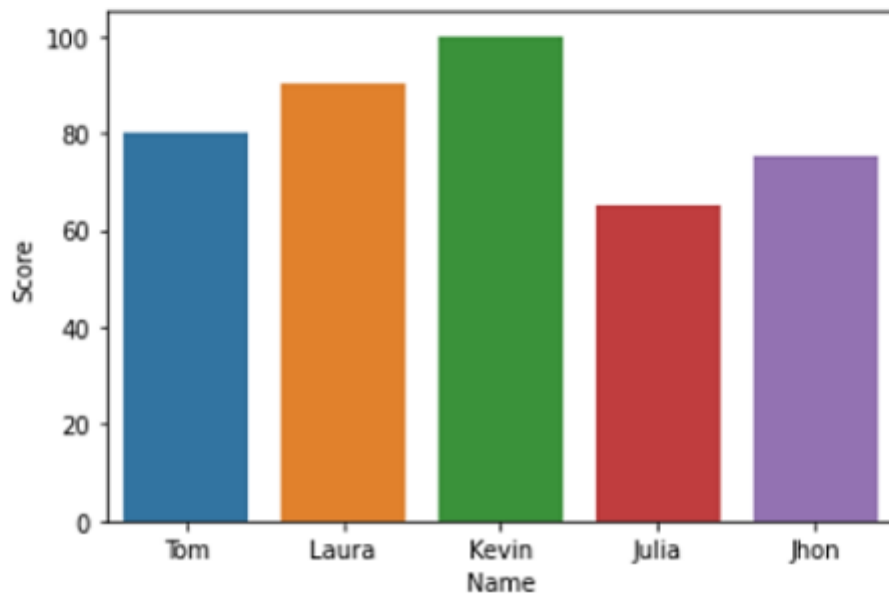
[함수 정의]

seaborn.barplot(x, y, data)

- x: x축 레이블
- y: y축 레이블
- data: 그래프 그릴 데이터

```
import pandas as pd
import seaborn as sns
df=pd.DataFrame({
    'Name':{0:'Tom',1:'Laura',2:'Kevin',3:'Julia',4:'Jhon'},
    'Score':{0:80,1:90,2:100,3:65,4:75}})
sns.barplot(x='Name', y='Score', data=df)
```

<matplotlib.axes._subplots.AxesSubplot at 0x7f9b82281850>



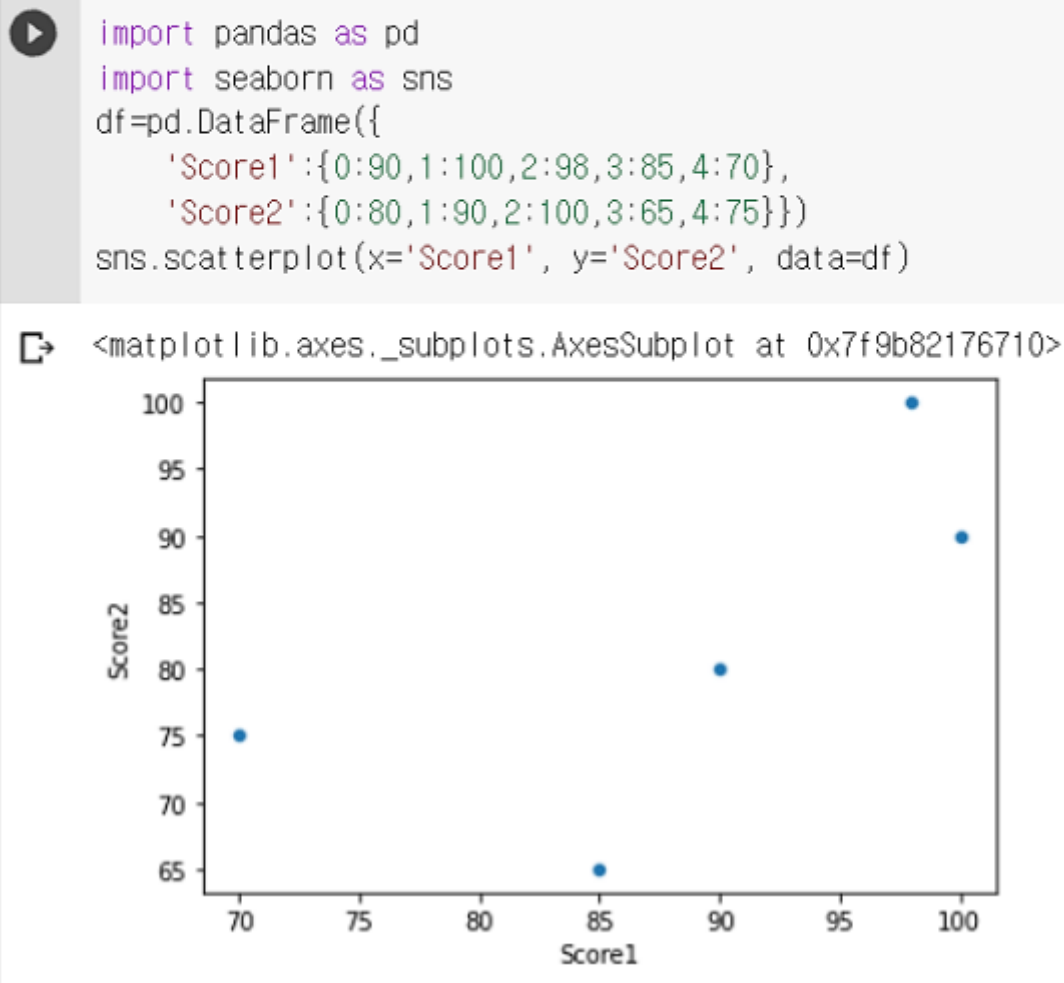
(6) 선점도 그래프 그리기

- scatterplot 함수 이용

[함수 정의]

seaborn.scatterplot(x, y, data)

- x: x축 레이블
- y: y축 레이블
- data: 그래프 그릴 데이터



3. 데이터 파일로 저장하기

1) CSV 파일로 저장하기

(1) to_csv 함수

- DataFrame에 저장된 데이터를 CSV 파일로 저장

[함수 정의]

DataFrame.to_csv(path_or_buf, sep, na_rep, columns, header, index, mode, encoding)

- path_or_buf: 파일 또는 파일 개체의 이름.
None이면 CSV 문자열로 변환. 기본값은 None
- sep: 구분 기호 문자열. 기본값 콤마(,)
- na_rep: 누락된 데이터를 나타낼 문자열
- columns: CSV 파일에 저장될 DataFrame의 열
- header: 열 이름으로 저장할 문자열.
False로 설정되면 열 이름을 파일에 저장하지 않음
- index: True이면 행 이름을 인덱스로 저장
- mode: CSV 파일의 모드. 기본값은 w
- encoding: CSV 파일의 인코딩 체계. 기본값은 utf-8

(2) DataFrame을 CSV 파일로 저장하기

```
import pandas as pd
df=pd.DataFrame({
    'Name':{0:'Tom',1:'Laura',2:'Kevin',3:'Julia',4:'Jhon'},
    'Gender':{0:'male',1:None,2:'male',3:None,4:'male'},
    'Age':{0:None,1:25,2:30,3:18,4:None}
})
print(df)
df.to_csv('./df_to_csv.csv')
```

	Name	Gender	Age
0	Tom	male	NaN
1	Laura	None	25.0
2	Kevin	male	30.0
3	Julia	None	18.0
4	Jhon	male	NaN

(5) DataFrame에서 특정 열만 선택하여 CSV 파일로 저장하기

클릭

```
import pandas as pd
df=pd.DataFrame({
    'Name':{0:'Tom',1:'Laura',2:'Kevin',3:'Julia',4:'Jhon'},
    'Gender':{0:'male',1:None,2:'male',3:None,4:'male'},
    'Age':{0:None,1:25,2:30,3:18,4:None}
})
print(df)
df.to_csv('./df_to_csv2.csv',
          columns=['Name', 'Age'],
          header=['user name', 'user age'])
```

클릭

더블 클릭

df_to_csv.csv X df >

총 5개 항목 중 1~5 필터

	user name	user age
0	Tom	
1	Laura	25.0
2	Kevin	30.0
3	Julia	18.0
4	Jhon	

페이지당 10 개 표시

2) JSON 파일로 저장하기

(1) to_json 함수

- DataFrame에 저장된 데이터를 json 파일로 저장

[함수 정의]

`DataFrame.to_json(path_or_buf, orient)`

- json 문자열로 변환. 기본값은 Node
- orient: json 형식 지정. columns(기본값), records, index, split, table, values 등

(2) DataFrame을 json 파일로 저장하기

```
import pandas as pd
df=pd.DataFrame({
    'Name':{0:'Tom',1:'Laura',2:'Kevin',3:'Julia',4:'Jhon'},
    'Gender':{0:'male',1:None,2:'male',3:None,4:'male'},
    'Age':{0:None,1:25,2:30,3:18,4:None}
})
print(df)
df.to_json('./df_to_json.jsopn')
```

클릭

df_to_json.jsopn X

```
1 [{"Name":{"0":"Tom","1":"Laura","2":"Kevin","3":"Julia","4":"Jhon"}}
```

클릭

sample_data

- df_to_csv.csv
- df_to_csv2.csv
- df_to_json.jsopn

더블 클릭

(3) DataFrame을 table 형태의 json 파일로 저장하기

```
▶ import pandas as pd
df=pd.DataFrame({
    'Name':{0:'Tom',1:'Laura',2:'Kevin',3:'Julia',4:'Jhon'},
    'Gender':{0:'male',1:None,2:'male',3:None,4:'male'},
    'Age':{0:None,1:25,2:30,3:18,4:None}
})
print(df)
df.to_json('./df_to_json2.json', orient='table')
```

3) Excel 파일로 저장하기

(1) to_excel 함수

- openpyxl 라이브러리가 설치되어 있어야 함
- DataFrame에 저장된 데이터를 excel 파일로 저장

[함수 정의]

`DataFrame.to_excel(excel_writer, sheet_name, header, index)`

- 데이터 프레임 데이터를 Excel 파일로 덤프
- sheet_name: 시트이름, 지정하지 않으면 sheet1로 설정
- header: 열 이름 포함 여부(True/False)
- index: 행 이름 포함 여부(True/False)

(2) DataFrame을 excel 파일로 저장하기

4) 여러 개의 DataFrame 저장하기

(1) ExcelWriter 함수

- 여러 개의 엑셀 시트를 갖는 엑셀 파일을 생성
- pandas.ExcelWriter() 객체를 생성한 후, to_excel() 함수를 이용하여 시트를 추가
- save 함수를 이용하여 저장

[함수 정의]

pandas.ExcelWriter(path_or_buf)

- path_or_buf: 파일 또는 파일 객체의 이름

(2) DataFrame을 excel 파일로 저장하기

```
import pandas as pd
df1=pd.DataFrame({
    'Name':{0:'Tom',1:'Laura',2:'Kevin',3:'Julia',4:'Jhon'},
    'Gender':{0:'male',1:None,2:'male',3:None,4:'male'},
    'Age':{0:None,1:25,2:30,3:18,4:None}
})
df2=pd.DataFrame({
    'Name':{0:'Tom2',1:'Laura2',2:'Kevin2',3:'Julia2',4:'Jhon2'},
    'Gender':{0:'male',1:None,2:'male',3:None,4:'male'},
    'Age':{0:None,1:25,2:30,3:18,4:None}
})
print(df)
excel_writer=pd.ExcelWriter('./df_to_excel2.xlsx')
df1.to_excel(excel_writer,sheet_name='info_data1')
df2.to_excel(excel_writer,sheet_name='info_data2')
excel_writer.save()
```

Pandas를 이용하여 데이터 분석하기

1. 탑승객 클래스별 생존율 구하기

1) 문제 분석하기

(1) 데이터를 확보하자

- seaborn의 titanic 데이터셋의 요소를 확인
 - Survived: 생존 여부, target 값. (0 = 사망, 1 = 생존)
 - pclass: 티켓 클래스. (1 = 1st, 2 = 2nd, 3 = 3rd)
 - sex: 성별
 - age: 나이(세)
 - sibsp: 함께 탑승한 형제자매, 배우자 수 총합
 - parch: 함께 탑승한 부모, 자녀 수 총합
 - embarked: 탑승 항구
 - Fare: 탑승 요금

(2) 데이터를 확인하자

- seaborn의 titanic 데이터셋을 읽어들임

- Pandas 패키지를 import함

```
import pandas as pd
```

- seaborn 패키지를 import 함

```
import seaborn as sns
```

- seaborn 패키지 내 titanic을 읽어들임

```
df=sns.load_dataset('데이터셋명')
```

- 누락 데이터를 확인

- isna와 sum 함수로 확인

```
df.isna().sum()
```

탑승객의 클래스(pclass)와 생존 여부(survived)에는
누락 데이터가 없음

survived	0	embarked	2
pclass	0	class	0
sex	0	who	0
age	177	adult_male	0
sibsp	0	deck	688
parch	0	embark_town	2
fare	0	alive	0
		alone	0
		dtype: int64	

- 탑승객 클래스별로 그룹화
 - groupby 함수 이용: 열별 그룹화

```
group_res=df.groupby(['pclass'])
```

- 생존여부가 0과 1로 되어있는지 확인
 - value_counts 함수 이용: 고윳값의 개수

```
df['survived'].value_counts()
```

- 클래스별 생존자와 사망자 수를 구함

- value_counts 함수 이용

```
group_res['survived'].value_counts()
```

- 클래스별 인원수를 구함

- count 함수 이용

```
group_res['pclass'].count()
```

- 클래스별 생존률을 구함

- lambda 이용

```
lambda x : x.survived.value_counts()/x.pclass.sum()
```

- 그룹 객체에 lambda를 적용

- apply 이용

```
res=group_res.apply(lambda x :  
x.survived.value_counts()/x.pclass.sum())
```

- DataFrame을 열방향으로 이동(왼쪽→오른쪽으로 재구조화)

- unstack() 이용

```
res.unstack()
```

Stacked (long)

index		
1	Col_1	10
2	Col_1	20
1	Col_2	30
2	Col_2	40



Un-Stacked (wide)

index	Col_1	Col_2
1	10	30
2	20	40

2) 코드 작성하기

(2) 데이터를 확인하자

```
▶ import pandas as pd
import seaborn as sns
titan = sns.load_dataset("titanic")
df=titan.loc[:, ['survived','pclass','sex','age','fare']]
group_res=df.groupby(['pclass'])
res=group_res.apply(lambda x :
                    x.survived.value_counts()/x.pclass.sum())
print(res.unstack())
```

```
↗
```

	0	1
pclass		
1	0.370370	0.629630
2	0.263587	0.236413
3	0.252546	0.080788

2. 연비와 차종 간의 선점도 그리기

제공된 data.zip 에서 auto-mpg.data-original 파일 사용

1) 문제 분석하기

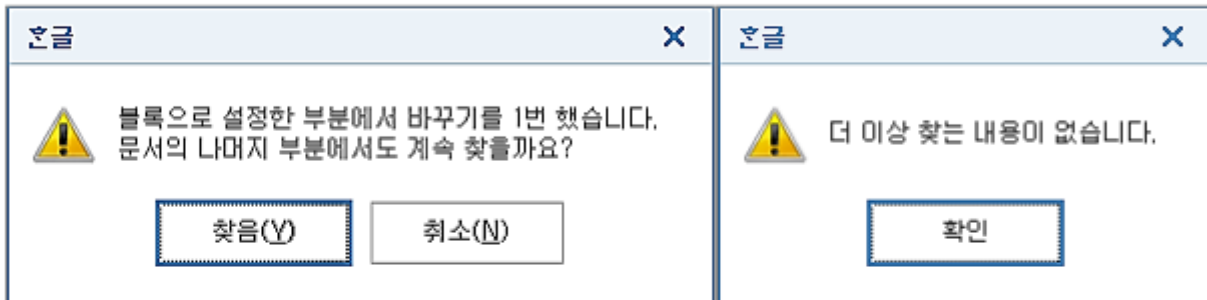
(1) 데이터를 확보하기

- 다운로드 받은 파일을 아래아한글로 열어 봄
 - 아래아한글의 "보기 " 메뉴에서 "조판부호"를 Check하여
각 항목의 구분자를 살펴보기

```
18.0 8 307.0 130.0 3504. 12.0 70 1 malibu ↵
15.0 8 350.0 165.0 3693. 11.5 70 1
18.0 8 318.0 150.0 3436. 11.0 70 1
16.0 8 304.0 150.0 3433. 12.0 70 1
17.0 8 302.0 140.0 3449. 10.5 70 1
```

- Header는 없음
- 각 항목의 구분자가 "제조국"과 "모델명" 사이만
Tab으로 구분되어 있음

- 다운로드 한 파일을 아래아한글로 열어봄
 - Tab을 공백문자 2자리로 치환하기
 - [모두 바꾸기] - [찾음] - [확인] - [닫기] 버튼 클릭
 - “더 이상 찾는 내용이 없습니다.”라는 메시지가 나올 때까지
찾음 버튼을 클릭



- 변경 결과 확인하기

```
18.0 8 307.0 130.0 3504. 12.0 70 1 "chevrolet malibu"
15.0 8 350.0 165.0 3693. 11.5 70 1 "buick skylark"
18.0 8 318.0 150.0 3436. 11.0 70 1 "plymouth satellite"
```

- 변경된 내용을 저장하기
- [파일] 메뉴 - [저장] 메뉴 - “파일형식 : 텍스트문서” 지정 -
[저장] - [저장] 버튼 클릭
- auto-mpg.txt 파일이 생성됨

- 파일을 읽을 때, Header를 아래와 같이 정의할 필요가 있음
 - 첫 번째 항목: mpg(연비)
 - 두 번째 항목: cylinders(실린더 수)
 - 세 번째 항목: displacement(배기량)
 - 네 번째 항목: horsepower(출력)
 - 다섯 번째 항목: weight(차중)
 - 여섯 번째 항목: acceleration(가속능력)
 - 일곱 번째 항목: model year(출시연도)
 - 여덟 번째 항목: origin(제조국 1(USA), 2(EU), 3(JPN))
 - 아홉 번째 항목: name(모델명)

- read_csv 함수 이용: text, csv 파일을 읽음

[형식]

```
pandas.read_csv(path, sep, encoding, names, na_values,
comment, skipinitialspace)
```

- path: 읽고자 하는 파일의 위치
- sep: 데이터 값이 구분되는 문자. 콤마(,)가 아닌 경우는 지정해야 함
- encoding: 파일과 python의 encoding의 설정이 맞지 않을 경우 지정. utf-8 또는 cp949로 설정
- names: 칼럼명을 변경해 줌
- na_values: 특정 값을 NaN으로 취급
- comment: 주석으로 분류되어 파싱하지 않을 문자 혹은 문자열
- skipinitialspace: 구분자 뒤에 있는 공백문자 무시 여부.
기본값은 False

- 각 항목의 구분자는 공백이고, Header가 없는 상태이므로 Header를 정의

```
column_names = ['MPG', 'Cylinders', 'Displacement',  
'Horsepower', 'Weight', 'Acceleration', 'Model Year', 'Origin',  
'Name']
```

- read_csv를 이용하여 읽어 보기

```
pd.read_csv(url, names=column_names, sep=" ",  
skipinitialspace=True)
```

- DataFrame의 plot 함수를 이용하여 선점도를 그리기

[형식]

DataFrame객체.plot(kind, x, y)

- Kind: 그래프의 유형을 지정
(line, bar, area, scatter, hist, box 등)
- X: x축 데이터
- Y: y축 데이터

2) 코드 작성하기

```
import pandas as pd
url = 'auto-mpg.txt'
column_names = ['MPG', 'Cylinders', 'Displacement',
                'Horsepower', 'Weight', 'Acceleration',
                'Model Year', 'Origin', 'Name']
raw_dataset = pd.read_csv(url, names=column_names,
                           sep=" ", skipinitialspace=True)
raw_dataset.plot(kind='scatter', x='Weight', y='MPG')
```

